COM3001 – Professional Project

Evaluating the effectiveness of different training techniques within the realm of Facial Emotion Recognition (FER)

Name: Omm Bhavsar
URN: 6726370
Supervisor: Dr Joey Sik Chun Lam

A dissertation submitted in partial fulfilment
of the requirements for the award of

BACHELORS OF SCIENCE IN COMPUTER SCIENCE

May 2025

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Decleration of Originality

"I confirm that the submitted work is my own work. No element has been previously submitted for assessment, or where it has, it has been correctly referenced. I have clearly identified and fully acknowledged all material that is entitled to be attributed to others (whether published or unpublished) using the referencing system set out in the programme handbook.

I agree that the University may submit my work to means of checking this, such as the plagiarism detection service Turnitin® UK and the Turnitin® Authorship Investigate service. I confirm that I understand that assessed work that has been shown to have been plagiarised will be penalised.

If in completing this work I have been assisted with its presentation by another person, I will state their name and contact details of the assistant in the 'Comments' text box below. In addition, if requested, I agree to submit the draft material that was completed solely by me prior to its presentational improvement."

Omm Bhavsar
May 21 2025

Abstract:

Facial Emotion Recognition attempts to bridge the gap between human-computer action by creating an architecture which can perceive and understand human emotions; this task can have several applications especially within the area of psychology, especially when mental wellbeing is involved. The project attempts to investigate the effectiveness of transfer learning and the creation of a custom CNN architecture to optimise an architecture for FER.

MobileNetV2, ResNet-18 and ResNet-34 were evaluated using two transfer learning strategies: training only the full connected/classifier layer; and training earlier convolutional layers in addition to the previous layer. The second approach consistently achieved superior results but introduced a moderate amount of overfitting. The individually trained architectures performed poorly when cross-set generalisability was tested. As a result, the datasets were compiled and used to train the architectures; even though performance remained on par with the second approach, overfitting was still apparent.

Therefore, a custom CNN architecture was created; the development process was iterative. Each revision of the architecture was optimised after the alteration of hyper-parameters, and the usage of a combination of loss functions (cross entropy loss [CEL]] and weighted cross entropy loss [WCEL]) and optimisers (Adam and SGD). The results from this section revealed the strengths of CEL paired with SGD; the trained architecture was the most stable and achieved the highest scoring metrics compared to other configurations. During later revisions of the architecture the Adam optimizer struggled significantly, often converging earlier than SGD but at a local minimum rather than global.

Table of Contents

Decleration of Originality 1
Abstract: 2
1 Introduction: 5
1.1. What is FER? 5
1.2. Research Question: 5
1.3. Real-World Applications: 5
1.4. Proposed Solution: 6
1.5. Research Objectives: 6
1.6. Scope of Research Project: 7
1.7. Beyond the Scope of Research Project: 7
2 Literature Review: 8
2.1. How do humans communicate their emotions? 8
2.2. How does modern facial recognition work? 8
2.3. Classic vs Deep Learning approaches for Face Detection: 8
2.3.1 Classic: 9
2.3.2 Deep Learning: 9
2.4. Classic vs Deep Learning approaches to FER 9
2.4.1. Classic 9
2.4.2 Deep Learning 10
2.5. CNN Architectural Comparison: 11
2.6. Dataset Imbalance 12
2.7. Lesson Learnt & Key Insights 13
3 Methodology and Implementation: 14
3.1. Dataset selection: 14
3.1.1. CK+ dataset: 14
3.1.2. JAFFE dataset: 15
3.1.3. KDEF dataset: 16
3.1.4. Other datasets: 16
3.1.5. Challenges of datasets 17
3.1.6. Justifications of choice: 19
3.2. Data Pre-Processing: 19
3.2.1. Data Augmentations: 19

3.3. Testing of an existing solution - DeepFace: 20
3.3.1. Testing Methodology: 20
3.3.2. Testing Environment: 20
3.3.3. Testing Implementation: 21
3.3.4. Testing Results 21
3.4. Transfer Learning Approach 22
3.4.1. Testing Environment: 22
3.4.2. Training Parameters: 23
3.4.3. Architectural alterations: 23
3.4.4. Layer Freezing: 24
3.4.5. Transfer Learning results: 24
3.5. Custom Architecture: 35
3.5.1. Testing Environment: 36
3.5.2. Architectural Variants: 36
4 Evaluation: 48
4.1. Results of the tested approaches: 48
4.2. Evaluation of approaches 48
4.3. Further work 50
5 LSEP Section: 51
5.1. Ethical Approval: 51
5.2. Data Usage: 51
5.3. Ethical Challenges: 51
5.4. Relevance to SDG: 52
References: 53

1 Introduction:

1.1. What is FER?

FER (Facial Emotion Recognition) is the task of identifying a person's emotion, based upon their facial expression; this can be done on a human-to-human basis, or as a HCI (Human Computer Interaction) using computer vision and deep learning techniques. Architectures are continuously being optimised to increase performance and improve generalisability. However, this is less prominent within the field of FER; it is a challenging task due to the constraints of the data. Emotions are expressed in different ways by humans, some emotions are clearly distinguishable (i.e., happiness and sadness) whilst others are not (i.e., fear and surprise). Many popular architectures used for object classification have shown clear signs of confusion adapted for FER.

1.2. Research Question:

What is the effectiveness of using transfer learning techniques vs a newly created architecture to improve the performance of FER using controlled datasets?

1.3. Real-World Applications:

Within a real-world setting, FER has several applications: for example, within a psychological field. For those who have autism, detecting and comprehending another person's emotions may be difficult. Having a system that utilises FER in real-time could possibly help them understand how these emotions are expressed, and how to similarly reciprocate them. In addition, FER could also be used to detect emotional distress for people who suffer from mental illnesses (i.e., depression, anxiety); a FER system could be paired with an AI assistant who could help a person to distress and return to a better state of mind.

However, there are many challenges that are prominent when working with FER; for example, class imbalance. An imbalance results in the model having favour towards the most prominent class, the accuracy of the model may remain high; however, it may struggle and/or fail to detect the less common emotions within the dataset. As a result, the model's generalisation capabilities within a real-world setting may be poor.

Another challenge that FER faces is dataset bias; in addition to class imbalance, the bias of humans within the dataset can affect the generalisation of the model negatively, due to possible underrepresentation. Certain datasets (i.e., JAFFE) are biased, due to only containing a subset of ethnicities. Training a model on a bias dataset could result in poor performance on others from different ethnicities (from the dataset). A possible solution could be to merge different datasets and compile them into a larger, more diverse set.

In addition, some FER datasets (i.e., JAFFE, CK+) are relatively small, consisting of only 213 and 903 images respectively. Training a model on a small dataset is possible, however it can lead to a large amount of overfitting; the model begins to only memorise the data rather than learn features from it. Once again, the model's generalisation performance would suffer substantially, due to a lack of variety with the data.

To minimise the negative effects proposed by these challenges, the research carried out will implement techniques of data augmentation, weighted loss functions, dataset compilation, and different training methodologies to explore how they affect a model's performance.

1.4. Proposed Solution:

The project wishes to investigate the effectiveness of transfer learning, and a custom CNN architecture (designed specifically for FER) which has been optimised using different loss function and optimiser configurations. The pre-trained architectures (MobileNetV2, ResNet-18 and ResNet-34) will be finely tuned using the datasets (CK+, JAFFE and KDEF) both individually and together (compilation of all the datasets). They will be subject to a cross-dataset tests to gauge whether generalisable or dataset specific features have been extracted. The custom architecture will be incrementally developed; additional layers will be added for feature extraction and regularisation in the hopes of improving the generalisation performance.

Studies which focus on FER evaluate architectural choices but fail to explore how different loss functions and optimisers affect performance; for example, they default to using the Cross Entropy Loss and SGD. As a result of this, this project aims to explore the effects of using a combination of different loss functions and optimisers. Specifically comparing the effects of using weighted cross entropy loss verses cross entropy loss; and SGD verse Adam.

1.5. Research Objectives:

1. Gather several datasets which can be used for classification. The datasets must have seven key emotions (happiness, sadness, fear, anger, disgust, surprise and neural), as to mirror the FAA feature provided by DeepFace [14].
2. Test the gathered datasets on pre-existing solution (DeepFace), record its accuracy and explain which emotions were the most mis-classified.
3. Evaluate the performance of different architectures (MobileNetV2, ResNet18, ResNet34) utilising transfer learning techniques.
    a. The datasets would first be trained individually on each model, and then test against the remaining datasets, to understand if the model can generalise well.
    b. The datasets would then be compiled for training and testing using each architecture.
    c. Compile and compare results of confusion matrix, accuracy, loss, precision/recall/F1-scores; explaining why certain architectures have a performance increase, etc.
4. Develop a custom CNN architecture incrementally; optimisation will be performed at each step.
    a. Evaluate performance and impact of different loss functions (weighted vs un-weighted), optimisers (SDG vs Adam) and hyper-parameter tuning (learning rate, momentum, weight decay).
    b. Implement strategies to improve training efficiency (i.e., early stopping and learning rate schedulers).
    c. Assess the model's effectiveness at classification using confusion matrices, loss graphs, precision/recall/F1-scores. Explain why architectural changes resulted in better/worse performance.
5. Tabulate all results from both transfer learning and fresh training on the custom CNN architecture and performing a comparative analysis.
6. **[Extension]** Further reinforce reasoning for architectural changes using Grad-CAM to visualise how CNN is analysis features and classifying them.
7. **[Extension]** Implement the feature to perform real-time emotion recognition using a video or webcam

The objectives mentioned depict the timeline of events, and how the project's life cycle will occur. All the objectives combined aim to develop an optimised FER model that can accurately classify facial emotions. The extension objectives are less prioritised within the current scope of this project, although if enough time is left after implementing the key features, they would provide further evidence of the robustness of the model created.

1.6. Scope of Research Project:

- The project must only use datasets which are publicly available (i.e., from Kaggle). They must only include participants who are over the age of 18.
- Transfer Learning must be evaluated and compared for the datasets.
- Custom CNN architecture must be developed, with reasoning to changes.
- Performance of using different loss functions and optimisers must be analysed with help from metrics (i.e., confusion matrix, loss graph, precision/recall/F1-scores).

1.7. Beyond the Scope of Research Project:

- Utilising GANs (Generative Adversarial Networks) to create more training data after analysing the current dataset. The utilisation of GANs could cause ethical concerns to rise due to the creation of data using people's faces.
- Deploying architecture into a real-time application.

2 Literature Review:

This section will cover researched literature that has used Computer Vision (CV) to classify emotions. The purpose is to review implementations of FER (Facial Emotion Recognition) models, analysing the features and techniques employed, which led to the researcher's success/downfall. These features and techniques will then be employed and tested, in a new CNN which will be created.

CV is the backbone of this project; it can be defined as a subspeciality "of artificial intelligence that uses machine learning and neural networks to teach computers … to derive meaningful information from … visual inputs …" by [1].

2.1. How do humans communicate their emotions?

Verbal communication dictates "relating to words" [2] by the "exchange of information" [3]; humans use this as the main method of communication. For example, we as humans converse with one another to socialize, whether that be in person or using a mobile phone. However, communication is not restricted to verbal only; as we talk, the muscles in our faces move, portraying the emotions that we are attempting to convey; this forms a facial expression. Non-verbal communication refers to "not using words or speech" [4]; humans communicate non-verbally using facial expressions. These facial expressions are identified as the "six basic universal emotions: happiness, sadness, anger, fear, surprise, and disgust" [5]. These expressions are understood by the receiving party via visual cues. For example, when 'happiness' is expressed: a person's mouth begins to form a stretched 'U' shape, their teeth become slightly visible; and their pupils dilate.

2.2. How does modern facial recognition work?

To understand how an emotion is recognised by the algorithm, we must first understand how the face is acknowledged. Using a commercially available solution DeepFace [14], we can see that a facial image is passed through a pipeline which detects, aligns and normalises the face. DeepFace gives the user the option to use classic or deep learning techniques to do the steps above. The detect step utilises an algorithm which can be chosen by the user (e.g., OpenCV, MTCNN, etc.) to create a "bounding box" around the face/faces present in the data. The align step ensures the image is straight, such that the features (i.e., eyes, mouth, nose, etc.) are parallel with the bounding box (classic and deep learning take different approaches). The normalisation stage utilises Dlib, which creates a mask of 68 points around the face and features (i.e., eyes, nose, mouth); this allows for noise and redundant data to be removed. For further analysis on the face (i.e., matching to existing image in dataset or emotion recognition), the mask is passed into a model (i.e., VGG-Face) which has been trained on a set of data relevant to the analysis.

2.3. Classic vs Deep Learning approaches for Face Detection:

To understand how the machine learning model interprets facial expressions, it first must have a face to analyse. In a real-time environment, a person's face will be juxtaposed in a setting with noise; for example, moving and background objects. As a result of this, it is important to have an algorithm which can extract only the needed information (facial data). There are several ways to extract this information from real time data; these approaches can be divided into two main categories, classic and deep learning.

Within the scope of this literature review, Viola Jones [6] will be reviewed as the primary technique for detection using a classic approach. This algorithm has been very popular in the past and the present, due to its speed and accuracy.

Regarding the deep learning approach, MTCNN will be the primary technique (a detection backend provided by DeepFace) [7].

2.3.1 Classic:

The algorithm works by translating the input image into a pixel intensity grid (each pixel given a value for intensity). An integral image is then created by performing summation on neighboring values; the creation of this image allows for Haar-like features to be extracted in a constant time (summation of neighboring values does not have to occur every time). During the feature extraction step, Haar-like features are identified using a sliding detection window. This window consists of 2-4 rectangle features which perform subtraction of pixel values from the integral image; the number of rectangles dictates the type of pattern which is being looked for. This step is repeated over the whole image, and with types of window sizes. During the training step, these features are then evaluated by AdaBoost which has decided a threshold value; as training occurs, this threshold value is updated and if the feature's value becomes less than, it is removed from the feature set. This step is repeated until a pre-defined number of features are selected. After training occurs, a cascade classifier is formed enabling the detection of faces.

Even though the training process for Viola Jones is classed as computationally expensive, the algorithm has a high detection accuracy. In addition, during a detection speed test, it was found that the algorithm was largely faster than pre-existing methods (RowleyBaluja-Kanade & Schneiderman-Kanade). However, the algorithm is not able to detect faces at a non-frontal orientation (need proof), largely due to the training of the algorithm being performed on front facing faces [6].

2.3.2 Deep Learning:

MTCNN (Multi-task Cascade Convolutional Neural Network) is an algorithm that employs deep learning techniques to detect faces in an image. The algorithm consists of four stages: P-Net to identify potential face regions; R-Net to filter out false positives; and O-Net to localize the facial landmarks; and finally, the regression and localization step that narrows in further on area occupied by the face detected.

The advantages of using a MTCNN are shown through the performance in varied conditions (i.e., lighting and different expressions); the accuracy of detection was said to remain consistently high [7]. [8] reinforces the advantage of MTCNN, as it has the best scoring metrics (precision, recall and f1 score) using a dataset of images with varying lighting levels. However, MTCNN has the longest inference time, most likely due to the logic of the algorithm; such that it utilises 3 nets and a regression/localization step.

2.4. Classic vs Deep Learning approaches to FER

This section will attempt to highlight the impact of classical approaches and deep learning approaches to FER. The approach to classification will be briefly discussed, followed by the analysis of some articles relating to approach. This analysis will aim to explain why each article performed as reported, and how possible improvements could have been made to better the results.

2.4.1. Classic

Before deep learning methods were popularized, there were other approaches taken to perform classification tasks. [9] shows several popular approaches that were taken to perform emotion analysis including SVMs. Figure 7 outlines the prominence of utilising SVMs for the task of emotion classification.

SVMs are not typically used for multi-class classification (but can be adapted to do so) and rather for binary classification; for example, the SVM can be trained to recognise a singular emotion against the rest. For this to happen, the data (images in this case) must first be pre-processed to have their most prominent features extracted (i.e., the image is turned into an n-dimensional array). The images are then “plotted” onto an n-dimensional graph, which shows some level of clustering [10]. The SVM algorithm then attempts to find the maximize the distance between both classes using the support vectors in each cluster [11].

[12] highlights the use of an SVM for emotion classification. The article tests the classification power of several classifiers (SVM, KNN, MLP); for training the classifiers, the CK+ dataset was used. The features from the image were extracted using HOG (histogram of gradients) and were reduced using PCA (principal component analysis). These features were then used to train the SVM, using 10-fold cross validation. This experiment yielded positive results; the SVM outperformed both KNN and MLP classifiers, achieving an accuracy of 93.89% when classifying 8 different classes of emotions.

Happiness, disgust and surprise produced the highest F1-Scores; these metrics indicate that the SVM is very strong at recognising features related to these emotions. However, the SVM is poor at recognising features for emotions: contempt and sadness. These results are most likely due to the class imbalance between emotions; as the contempt class only contains 18 images. The SVM is unable to recognise the features that correlate to being contempt, as it has learnt more about another image class.

To further develop the approach taken, the SVM could be trained using a compilation of several controlled datasets; for example, an inclusion of JAFFE and KDEF. The addition of these datasets would help reduce class imbalance, and aid in the development of the model's robustness (ability to classify unseen data).

As mentioned above, [9] is another article which provides an in-depth analysis of the usage of SVM’s for FER. The study compiles the results from many different classical and NNB (Neural Network Based) approaches, referencing the dataset used and its accuracy. SVMs can be viewed as strong performers as they output an accuracy above 70%.

However, the article does have some shortcomings; for example, it is stated that classic approaches, such as SVMs have “very high specificity and low generality to solve the emotion recognition problem”. This suggests that the classic approaches struggle adjusting to images which they were not trained on. Unlike [12], the article does not mention other metrics (i.e., precision, recall, F-Score), and only refers to the approach's accuracy. If these metrics were included, it would aid in the recognition of how the approach reacts to the data. Performance relating to classification rates and class bias could be discovered.

2.4.2 Deep Learning

Deep learning techniques have gained popularity throughout the years, Figure 1 within [9] shows this trend. This increase in popularity could be attributed to the deep learning approaches strengths in better generalisation.

[13] reviews the use of lighter weight CNN architectures to perform FER. The article explores how transfer learning and architectural changes can be leveraged with their implementation (CLCM) of a CNN (using MobileNetV2 as a framework). Their methodology involved freezing the lower layers and only training the final parts of the network; this process of fine-tuning allowed for prior weight values to be updated whilst retaining the model’s ability to classify certain key features (i.e., edge detection). Using transfer learning resulted in performance metrics being better than/on par with existing models such as MobileNetV2 and ShuffleNetV2.

[5] outlines the creation of a newly proposed CNN architecture called GIMEFIVE. To train and test the model, several datasets (i.e., RAF-DB, FER2013, etc.) were compiled together; this resulted in a wider range of data that could help in generalisation. To ensure that the model could understand the data, Grad-CAM was implemented; this allowed for a visual representation of how the CNN views data to be understood. The article covered how different architectures affect the model's overall performance, but didn't cover how the model could be further optimised using different loss functions, optimiser and hyper-parameters.

A drawback the GIMEFIVE article faces is the computational cost. Compared to the CLCM architecture mentioned in [13], the GIMEFIVE model is larger with a parameter count that is x4 larger. However, this increased complexity allows for the model to be more accurate in classification, as more data can be learned during the training process.

DeepFace [14] is currently an existing solution that primarily focuses on facial detection, whilst also offering the feature of facial attribute analysis (FAA). An element of FAA is the emotion classification, DeepFace offers the ability to perform FER within a real-time setting.

A naive approach could be taken, using a simple CNN architecture; however, the performance of that model would suffer substantially. The CNN would not be complex enough to capture key features to correctly classify an emotion and would rather be "guessing". Other solutions such as the one shown in [15] have also been created. This model utilises a shallow CNN architecture, and the FEC dataset to classify emotions. The model classifies emotions; however, it does so with minimal accuracy. It states that it has an accuracy of approximately 57% on the testing dataset, whilst obtaining 92% on the training dataset with a loss 10x lesser than that of the test set. This shows the model has overfit to the training data and has memorised.

2.5. CNN Architectural Comparison:

Throughout these years, there has been a large increase in the use of CNNs for classification tasks, due to their power of feature extraction. As a result of this, many different architectures have been developed, all with their varying strengths and weaknesses. This sub-section will focus on literature that explores the evolution of these architectures, with comparisons of their strengths and weaknesses.

[16] reviews the architectural developments of CNNs used for image classification; ResNet is one of these. ResNet was popularized after winning the ILSVRC competition, due to its ability to mitigate the vanishing gradient problem.

As traditional network trains, they follow a sequential flow, the input is passed through every layer. As the network grows deeper for the extraction of features, the gradient begins to decrease to almost zero value. This decrease can cause premature convergence. ResNet combats this effect by implementing Residual Blocks with skip connections, they allow the network to keep learning features at each level, even if the previous layer was not able to extract anything meaningful. Implementing these techniques enabled ResNet to grow into a deeper network; the increase in depth enables the network to learn more complex features.

[5] highlights the strong performance of GIMEFIVE which uses two different architectural combinations. In particular, the architecture which utilises dropout and batch normalisation; it performs the best in terms of accuracy on the testing and validation sets of each dataset splits. Due to the lack of variation between the accuracy metrics, it can be inferred that no overfitting occurred during the training process, indicating the use of dropout was effective.

The architecture utilises four convolutional layers, each followed by batch normalisation, max pooling and a dropout. The final layer is somewhat similar, however instead of dropout after the max pooling, average pooling follows with the

introduction of a linear layer. Model depth is a separating feature; for example, ResNet-18 is slightly deeper than GIMEFIVE. ResNet-18 utilises 17 convolutional layers, whilst GIMEFIVE only uses 5. GIMEFIVE will output a greater number of feature maps; the architecture will be able to learn more about the data, whether that be finer edges signifying wrinkles around the eyes or nose. Even though residual blocks in theory should improve performance, when it was used with the GIMEFIVE architecture, it resulted in worse performance.

Both [5] and [16] are quite computationally expensive, such that they have over ~10 million parameters each; if the architectures were to be used in a real-time setting, higher end hardware would be needed to ensure a faster inference time. [13] address this issue in another approach taken to FER – CLCM; this architecture has ~2.3 million parameters and was based off MobileNetV2. The architecture utilises similar features to GIMEFIVE such as dropout, and batch normalisation.

The accuracy of model classification was tested across the listed architectures using the RAF-DB and FER-2013 dataset. The architectures all performed similarly to one another, indicating that even though the CLCM model is lighter weight, it still holds a strong level of classification power. The results are as follows:

Table 1 - Accuracy across RAF-DB and FER-2013 datasets

Dataset	Architecture	Accuracy (%)
RAF-DB	ResNet-18	81.3
	GIMEFIVE	86.5
	CLCM	84.0
FER-2013	ResNet-18	64.7
	GIMEFIVE	67.5
	CLCM	63.0

2.6. Dataset Imbalance

When training an architecture to perform FER, the dataset is extremely important. In a perfect scenario, the dataset would be large and be equally distributed; each class would have the same amount of data. However, this is not the case with many FER datasets; most datasets show some sort of class imbalance, such that some classes (i.e., happiness and sadness) are always overrepresented, and others (i.e., fear and disgust) are underrepresented. The FER GIMEFIVE dataset in [5] is an example of this; for example, the disgust class only has 4100 images whilst happiness had 18246, an amount which is almost 4.5x larger. This imbalance caused the architecture to get confused; disgust was only classified correctly 61% of the time, often getting misclassified with sadness and anger. The lack of images within the disgust class hindered the architecture's ability to classify it; features specific to disgust were either overwritten or ignored due to overrepresented classes making more frequent weight updates throughout training.

Reviewing the confusion matrices in [13] further highlighted the negative impact of a skewed dataset; the MobileNetV2 architecture trained on FER-2013 achieved 0% accuracy on the disgust class and was mostly misclassified as anger (44%) and sadness (42%). Once again, the architecture was more suited to learning the features from classes which share similar features to disgust, rather than learning distinctives features relating only to disgust.

2.7. Lesson Learnt & Key Insights

The literature highlighted several ways in which FER could be implemented. For this project, deep learning was chosen to implement FER. [5], [13] and [16] highlighted some factors which were taken into consideration when developing the project's methodology section.

Due to success of transfer learning with [13], an attempt was made to perform FER using several pre-trained architectures (i.e., MobileNetV2, ResNet-18, ResNet-34). Two approaches to transfer learning were devised: the first involved only unfreezing the final layer in the architecture (fully connected layer/classifier head); the second involved unfreezing some earlier convolutional layer, to allow for FER specific features to be extracted and learnt. To test whether the features were meaningful, a cross-set generalisation test was developed; the architectures would then be tested against another dataset, similarly to how [13] tested the architecture against the CK+ dataset.

GIMEFIVE [5] highlighted the strengths of creating a custom architecture specifically for FER, rather than utilising pre-existing CNN architectures; hence due to its improved performance, an experiment of similar nature was proposed. The experiment would involve creating a custom architecture and incrementally improving it via the addition of extra layers, regularisation techniques and different loss function – optimiser pairings. This newly developed CNN hoped to be lighter weight and perform on par with DeepFace or its transfer learning counterparts.

[Section 2.6] highlighted some concerns surrounding the use of an imbalanced dataset; Section 3.1 will highlight some of the approaches taken by other researchers to combat this imbalance.

3 Methodology and Implementation:

This section of the report will investigate how: datasets, training approaches, neural network architectures, optimisers and loss functions affect the performance of a FER model.

3.1. Dataset selection:

To train the CNN architecture, a dataset containing images representing facial expressions was needed. As mentioned in [Section 1] the datasets must contain 7 emotions: Anger, Disgust, Fear, Happiness, Sadness, Suprise, and Neutral.

After reviewing the literature from [Literature Review] and performing an additional search for FER datasets, three datasets were chosen: a subset of Cohn-Kanade (CK+) [17], JAFFE (Japanese Female Facial Expressions) [18], and KDEF (Karolinska Directed Emotional Faces) [19]. All images in the datasets were taken in a controlled environment, using lighting where necessary to reduce noise and other inconsistencies.

3.1.1. CK+ dataset:

The CK+ dataset was chosen due to it being referenced in several pieces of literature; however, the full dataset containing both images and videos was not available on Kaggle and accessible for public usage. As a result of this, a subset of CK+ that contained 920 images was used. This dataset was then further reduced to remove the contempt emotion, resulting in only 902 facial images. The data is split as follows:

Table 2 - Shows the number of samples across each class within the CK+ dataset

Emotion	Number of samples
Anger	45
Disgust	45
Fear	25
Happiness	69
Sadness	28
Suprise	83
Neutral	593

The CK+ dataset differed in representation in comparison to bother JAFFE and KDEF; instead of having a folder structure of images in folders, a csv file was used. The csv displayed a table: emotion, pixel and usage were the column titles.

- Emotion: an index relating to an emotion class (i.e., 0: Anger, 1: Disgust, etc.).
- Pixel: an array of size 48x48 which stored a value for each pixel in the image.
- Usage: the set which the image belonged to (train, public test and private test).

The data from the csv was split into a ratio of 8:1:1, such that: 80% of the data was provided for training, 10% was provided for validations (public testing), and 10% was provided for testing (private testing). The training set had a total of 720 images; the validation set had a total of 93 images, and the testing set had a total of 89 images.

For the purposes of the project, a folder structure containing the images in their given class needed to be created. A script was developed to transform the 48x48 pixel array into images to be stored in their respected class folder. The script utilised data frames (provided by pandas), to gather all the data into a single table object. This object was then

filtered based on usage (train, public test and private test), which returned a data frame containing all images related to that usage. The emotion class and pixel arrays from these data frames were then abstracted. Using PIL (Python Imaging Library) enabled the array of pixels to get transformed into 48x48 images which could then be saved in the given emotion class folder. This process was repeated for each usage set.

Drawbacks:

- Bias in diversity; it is not inclusive of all races.
- Class imbalance in emotions; the neutral class has 593/902 images.
- The dataset is quite small, only 902 images in total.

These aspects would negatively impact a model during training. Firstly, the class imbalance would lead to the model learning features better from the prominent class (neutral), it'd be able to easily distinguish neutrality, but could possibly struggle to classify fear. Furthermore, the bias in data diversity could lead to poor model generalisation, the model may not be able to adapt to new subjects/images. As well as that, the dataset is quite small compared to other popular FER datasets (i.e., FER-2013); this could negatively affect model training, causing the model to memorise the training data rather than learn key features from it.

3.1.2. JAFFE dataset:

The JAFFE dataset was chosen due to it being mentioned in several papers [Literature Review]. The dataset was found and publicly available on Kaggle. The dataset contained 7 images folders relating to a certain emotion and had approximately 30 images in each folder. The dataset had a total of 213 images, and is split as follows:

Table 3 - Shows the number of samples across each class within the JAFFE dataset

Emotion	Number of samples
Anger	30
Disgust	29
Fear	32
Happiness	31
Sadness	31
Suprise	30
Neural	30

As no training/validation/test split as given, a script had to be developed to perform this action. As the CK+ dataset utilised a split of 8:1:1, it was decided that the JAFFE dataset would follow the same path, to remain consistent. The script randomly selected images based on the percentage split in each folder and then moved them into a specific split folder based on emotion type. This action was performed using the shutil library provided in Python.

Drawbacks:

- Bias in diversity; it is not inclusive of all races and genders (only Japanese Females).
- The dataset is extremely small, only consisting of 213 images in total.

Once again, bias in the diversity of training data would negatively impact the model's ability to generalise and adapt to new data, due to only being trained on a specific subset of participants. The extremely small size of the dataset is again a large issue; the model would begin overfitting the data and begin memorising it.

3.1.3. KDEF dataset:

The KDEF dataset was chosen after attempting to find more FER datasets on Kaggle which had participants over the age of 18. The dataset is constructed in a similar manner to JAFFE regarding initial folder structure. The dataset is composed of emotions shown by 35 female, and 35 male actors. Each emotion class folder has approximately 420 images in each, the split is as follows:

Table 4 - Shows the number of samples across each class within the KDEF dataset

Emotion	Number of Samples
Anger	420
Disgust	420
Fear	420
Happiness	420
Sadness	419
Suprise	419
Neural	420

Similarly to JAFFE, no training/validation/test split was given. The dataset underwent the same splitting procedure as mentioned in the JAFFE section.

Drawbacks:

- Bias in diversity; it is not inclusive of all races and genders

Once again, the dataset is biased due to only including participants within a singular ethic group; following similar reasoning to JAFFE, we can see why this would be an issue.

3.1.4. Other datasets:

After reviewing the literature from [Literature Review – Approaches to FER], there were several other frequently used datasets. [5], [9], [13] highlighted the popularity of the following datasets: AffectNet, RAF-DB, and FER-2013. These datasets have gained popularity due to their large size, and range of diversity. The sizing breakdown is as follows:

Table 5 - Shows the number of samples across each class within the FER-2013 and RAF-DB datasets

	Dataset (samples)	
Emotion	**FER-2013**	**RAF-DB**
Anger	4953	867
Disgust	547	877
Fear	5121	355
Happiness	8989	5957
Sadness	6077	2460
Surprise	4002	1619
Neutral	6198	3204

Reviewing the distribution of data across both datasets shows FER-2013 is more balanced than RAF-DB; for example, all classes excluding Disgust have number of images that are within a similar range. Training the model using the FER-2013

class would result in a model that could hold a better classification power, due to its even distribution of images across classes. The RAF-DB model would perform worse within the fear, anger and disgust classes due to the lack of training data.

FER-2013's most under-represented class is disgust; if a model were to be trained using this dataset, it would struggle in classifying it. The other classes in the dataset would have precedence over it, and its features would not be learnt to the same extent as the larger classes. The same reasoning can be applied to the fear, anger and disgust classes of RAF-DB.

These datasets stand out from CK+, JAFFE, and KDEF, due to the conditions that the images were taken in; for example, varied lighting conditions. The images in these datasets have been taken in the wild, and not in a studio setting. This results in there being more noise and a chance of obstructions within the images. Even though there may be more noise, if the model were to be trained on these datasets, it would have a greater ability to generalise newly seen data; this is due to the large number of samples within each dataset. In addition, the model could hold a lesser bias such that the training data is not skewed to a certain subset of ethnicities.

3.1.5. Challenges of datasets

This section will attempt to address the challenges faced by using imbalanced datasets, giving an insight into how this project attempts to employ any mitigation strategies. In [Figure - n], the distribution of images on a per class basis can be viewed, along with an individual breakdown:

Figure 1 - Shows the image distribution of each dataset on a per class basis

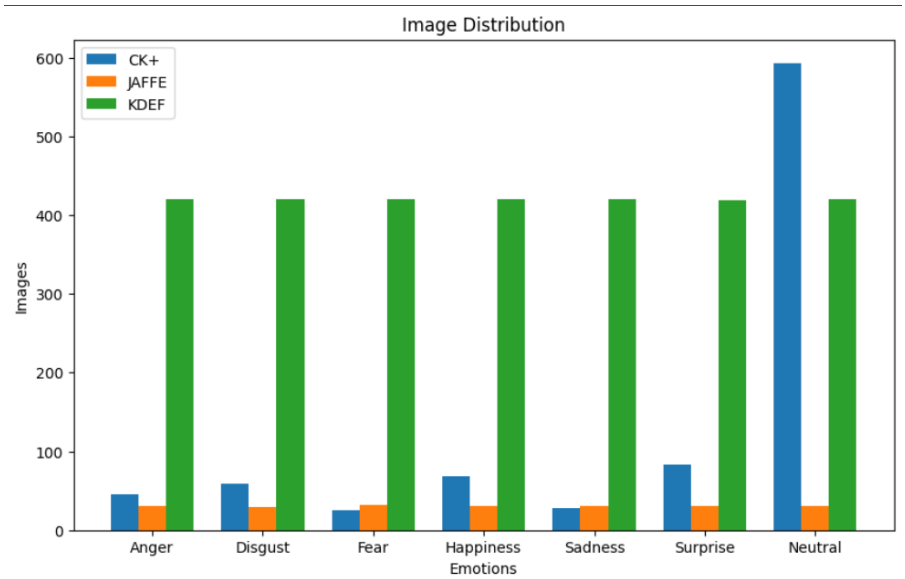


Table 6 - Shows the total number of samples across each emotion class after dataset compilation

Emotion	Total number of samples
Anger	495
Disgust	508
Fear	477
Happiness	520
Sadness	479
Surprise	532
Neutral	1043

[Figure 1] and [Table 6] show the skew of data towards the neutral class; for example, every other class has between 480 and 530 images. The neutral class almost doubled that amount with a total of 1043 samples. This uneven skew of data would cause the model to learn about features relating to neutrality more than any other class, inadvertently impacting the model's performance and ability to generalise.

To combat this, some countermeasures were investigated some of which were highlighted in [20]:

- SMOTE (Synthetic Minority Oversampling Technique) - [21]
  The algorithm works by operating on the feature space rather than data space. The k-th nearest neighbors are chosen from each sample in the dataset, and a new sample is generated by interpolating between the sample and its nearest neighbors. The new data will have features and characteristics that are like the originals.

  However, several issues are presented when using SMOTE:

  - Further exemplified bias – if the dataset is skewed to a certain demographic, SMOTE would only create images related to that demographic. Even though there are more samples in the dataset, it will not necessarily be diverse.
  - Increased error rate – newly generated samples may have an increased chance of overlapping features between certain emotion classes. This would further confuse the model and reduce classification accuracy.
  - Ethical concerns – the newly generated data would be using information from participants within the dataset, their consent for generating synthetic samples may not have been explicitly stated.
- Random Resampling - Over Sampling
  [22] refers to oversampling as a “naïve technique” to achieve better class distribution. Over-Sampling involves selecting random images within the imbalanced class and duplicating a subset of them until the target class has enough images within it.
  In this scenario, every class other than neutral would undergo oversampling. This would enable all classes to be represented equally, such that model does not favour one class over another.

  However, oversampling introduces issues:

  - Overfitting – as the images in the dataset are duplicated, the model may begin to memorise the features in duplicated images, rather than learning and understanding them.
  - Increased bias – as no new data is being created and existing data is being duplicated, the existing bias is increased. This would worsen the model’s ability to generalise.
- Weighted Loss Function
  [23] highlights the use of the weighted loss function to address an uneven class distribution. It works by assigning a set of penalties to each class; classes that are under-represented are given a higher penalty. [23] discusses that it “makes the model more sensitive”, ensuring the class is treated equally to the larger classes.

  However, the weighted loss function can introduce some issues:

  - Lack of generalisation – the loss function cannot help the model learn features from the minority class; it can only penalise its errors more. No additional data is being created/added to the set; the model is not learning about the class more.

3.1.6. Justifications of choice:

Even though the other datasets listed in [Section 3.1.4] have their advantages over the selected datasets (CK+, JAFFE, KDEF), they had to be disregarded. This was due to the ethical concerns surrounding them; the images from the dataset were scrapped from the internet, hence no formal consent was given for them to be taken and/or used. In addition, the dataset also contains participants who were under the age of 18.

The weighted loss function was chosen as a possible solution to combat the imbalance problem: SMOTE was disregarded due to ethical concerns; and random sampling was disregarded due to the bias it would impose on the architectures.

3.2. Data Pre-Processing:

Each of the datasets mentioned in the section prior all have different size and channel depth; for example, CK+ and JAFFE both are black and white images, whilst KDEF is color. When training a machine learning model, it is best practice to ensure all the data is of the same size and color. If the data were to be compiled together without pre-processing, the model's performance would be hindered, due to mismatched sizing. As a result, the model could learn noise or other features which are not relevant to emotion classification.

The size of each image in the datasets is as follows:

- CK+: 48x48 pixels.
- JAFFE: 256x256 pixels.
- KDEF: 562x762 pixels.

To stop the model from learning noise, and hopefully improve its classification ability, the images in the dataset are first passed through an instance of an MTCNN. As mentioned in the [Literature Review], MTCNN is used to detect faces in images. After an input image is fed into the MTCNN, a bounding box is created around the face; this box is then resized to 224x224 and saved as a new image into the dataset. The instructions for carrying this operation out were given in [24], and the code was forked from [25].

The images in each dataset were resized to 224x224 pixels, due to it being the default input size of images that were used to train MobileNetV2, ResNet-18 and ResNet-34. A smaller size could have also been used, but the input layer of the model would have to be altered, which would possibly affect the model's ability to classify.

3.2.1. Data Augmentations:

To separate the data on a split basis (training, validation and testing), a class was created to store the images. The images in the class were stored using ImageFolders provided by torchvision; the images in the train class then underwent transformations (a set of augmentations) which are as follows:

- Resize (224x224) - resizes images to 224x224 pixels if they are smaller/larger. This ensures that all data is of the same size before being fed into the network, reducing the likelihood of noise being learned.
- Grayscale (3) - converts the image to grayscale in each channel (Red, Green, Blue). This ensures all data is the same colour before being fed into the network.
- Random Horizontal Flip (p = 0.5) - randomly flips the image horizontally with a probability of 0.5. Adds in an element of randomness and creates variation in training data; helps combat overfitting, as data is different to rest.

- Color Jitter (brightness = 0.2, contrast = 0.2) - changes the brightness and contrast of the image. Altering the lighting level within the images may help exaggerate some features further and reduce the impact of less relevant features.
- Gaussian Blur - applies a gaussian blur to the image. This reduces noise in the image, and aids in the prevention of overfitting.
- Random Rotation - randomly rotates the image 10 degrees. The model is forced to recognise features that are in different positions, rather than memorising positions.
- Random Resized Crop - randomly crops and image and resizes it.
- ToTensor() - converts image into a tensor, which can be sent to data loaders.
- Normalisation - normalises the images based on set of values.

These augmentations to the data allow for the dataset to have more variety, and mirror such that would occur in a real-world scenario. In addition, the application of these augmentations allows for some prevention in overfitting. The model is forced to learn the key features that relate to a certain emotion, rather than memorising the data.

The data in validation and testing sets needs to remain un-augmented; the model's performance is gauged using this data. The data needs to remain untouched, to show the model's ability to react to unseen data. No additional augmentations were applied other than the ones listed below:

- Resize (224x224)
- Grayscale (3)
- ToTensor()

For selecting random images from the datasets when creating training, validation and test splits, the seed of 42 was selected during the randomisation process. Seeding allows for reproducibility and ensures that if the data needed to be refreshed, it would be the same selection every time.

3.3. Testing of an existing solution - DeepFace:

3.3.1. Testing Methodology:

Before implementing a custom approach to FER, testing was performed on the existing solution DeepFace. The testing involved feeding all images in the compiled dataset into DeepFace, recording the number of correct and incorrect classifications; enabling an accuracy metric to be gained. The decision was made to test the training, validation and test splits separately, rather than compiling them into a singular set; this allows for a greater comparative analysis between DeepFace and the later models.

3.3.2. Testing Environment:

Due to dependency issues on the Otter Lab Machines, Google Colab needed to be used to test the accuracy of DeepFace. The lab machines had an installation of Python which was not compatible with the needed TensorFlow dependency used by DeepFace. To abide with the ethical concerns of the project, the dataset was uploaded as a zip file into the runtime's environment; this meant that after the script's runtime was completed, the environment variables were deleted.

The runtime environment used is as follows:

- Runtime Type: Python 3
- Hardware Accelerator: T4 GPU

3.3.3. Testing Implementation:

Firstly, the dataset was uploaded to the script's runtime environment as a zip file. This zip file was then extracted using the **zipfile** package provided by Python. This resulted in the extraction of the images in the training, validation and test sets.

As each dataset split was being tested separately, a class was made to encapsulate each dataset. The class contained a dictionary of emotion labels; each label mapped to the output which DeepFace gave after processing each emotion. For example, when the script tested the emotion of surprise, it outputted:

- {'surprise': 101, 'fear': 75, 'disgust': 0, 'happy': 33, 'sad': 37, 'angry': 79, 'neutral': 100}.

The output shows how DeepFace viewed the images in the surprise folder, it was able to classify surprise with an accuracy of 23.76%.

The test_dataset function was developed to automate the process of testing each in the relevant dataset split folder. The function iterates through each emotion folder and passes in each image in the folder into DeepFace's FAA method; a predicted emotion is returned. This prediction is then tested against the actual emotion label; this process is repeated for every image. The classifications are then combined into a single nested dictionary, and the accuracy of each emotion is then calculated using the metrics from correct and incorrect predictions.

3.3.4. Testing Results

After passing in the dataset for testing, the output results are as follows:

```
-----------------------------------------------------------------
47.05% Accuracy
-----------------------------------------------------------------
      index  surprise   fear  disgust  happy    sad  angry  neutral
0  surprise     23.87  18.23     0.00   7.52   8.83  17.86    23.68
1      fear      3.35  11.74     0.42  17.40  24.11  23.69    19.29
2   disgust      0.00   3.74     5.71  15.75  15.94  52.76     6.10
3     happy      0.19   1.15     0.00  93.85   2.88   0.38     1.54
4       sad      0.00   2.72     1.88   4.18  61.09  12.55    17.57
5     angry      0.00   3.03     1.62   3.03  22.42  53.13    16.77
6   neutral      0.00   1.34     0.00   2.68  24.35   9.11    62.51
```

Figure 2 - Shows the confusion matrix of accuracy from DeepFace

The emotion class with the best accuracy was happiness. Reviewing the metrics table shows happiness scored above 90% on classification accuracy; this indicates that the DeepFace model is confident at identifying it. In addition, the mis-classification accuracy is minimal, with the model having slight confusion with other classes: sadness and neutrality.

The model has some ability to classify sadness and neutrality; the two classes have the second and third highest accuracy metrics. However, the performance is not comparable to happiness, only scoring between 61% - 62%. Sadness is confused with neutrality, and neutrality is confused with sadness; this symmetric confusion may be due to the emotions sharing very similar features to one another.

DeepFace struggles significantly with detecting disgust, fear and surprise. These three emotions have the lowest scoring accuracies, with disgust achieving the lowest accuracies across all testing splits. DeepFace struggles recognise and classify disgust; reviewing the misclassifications shows disgust is mistaken for anger, sadness and happiness. During the training process of the model, there may have been a class imbalance which led to this underrepresentation.

In addition, DeepFace may be struggling to interpret emotions which share features with one another; for example, surprise and fear. Both emotions are presented with similar features: the widening of a person's eyes and mouth, the raising of their eyebrows. The model may not be complex enough to identify other key features that humans can detect; for example, such as pupil dilation compounded widening of the eyes and eyebrow orientation.

For some emotions there is an overlap in the misclassification categories. In the case of disgust, the three most common emotions that were misclassified were: anger, happiness and sadness. Disgust can be broken up in a few key features: slight raising of upper lips, wrinkling around the nose, upward tilt of eyebrows and the narrowing of the eyes. DeepFace may link the raising of the upper lip as an indication of happiness; the upward tilt of eyebrows as sadness, and the narrowing of eyes as anger. These features confuse DeepFace, and lead to an overall mixed classification accuracy.

Reviewing the accuracies suggests that the DeepFace model underperforms. The model achieves an average accuracy of 47.05%. The model struggles significantly to identify emotions which are not happiness. This may be a result of an imbalance in the training dataset; the model learnt more features relating to its best performing emotions, rather than its worst. The uneven distribution of images in each emotion class could have resulted in this underrepresentation. However, the training dataset used for DeepFace could not be located, hence this is speculation.

3.4. Transfer Learning Approach

This section will cover the use of transfer learning approaches to FER using the following architectures: MobileNetV2, ResNet-18, and ResNet-34. These models were chosen due to their popularity within several articles covered in the [Literature Review Section]. This section aims to test whether transfer learning is a viable approach to FER, and whether it can produce a finely tuned architecture which can generalise to new data.

To test whether the architectures can generalise, domain adaptation will occur [26]. Domain adaptation involves testing a trained model on another dataset to assess whether the model has learnt generalisable features or dataset specific features.

However, due to the size and differing domains of the datasets, a prediction has been made for the lack of generalisability the individual models will show. As a result, all the datasets will be compiled into one, allowing for a training set with more diversity. The goal of this is to see how the model reacts to a range of data, and whether it can learn how and where the features differ. Compiling the datasets aims to produce an architecture which is robust and has a greater ability to generalise.

3.4.1. Testing Environment:

The models were trained and tested using the Otter Lab Machines provided by the university. Google Colab was not used in this scenario, as it did not have enough processing power to train the models at an efficient rate. The hardware and software specifications are as follows:

- Hardware:
    - CPU: Intel Core i9-13900
    - RAM: 64GB
    - GPU: NVIDIA RTX A4000, 16GB GDDR6 VRAM
- Software:
    - Visual Studio Code
    - Python 3.11.7 – using PyTorch

3.4.2. Training Parameters:

- Optimiser: Adam
    - Learning Rate: 0.00001
    - Weight Decay = 0.01
- Loss Function: Cross-Entropy Loss
- Epoch: 100
- Batch Size: 16, num_workers = 8

The training parameters for the transfer learning approaches can be seen listed above. Within this section, only the Adam optimiser, and the Cross-Entropy Loss function are tested. The primary selection of these was due to the time constraints placed on the project; training all the models using both optimisers (SGD and Adam) and loss functions (weighted and unweighted) would result in an extended training time.

[27] highlights the strengths of the Adam optimiser. Adam achieves the best results: lowest loss and highest accuracy across both training and validation phases. In addition, Adam achieves "quick learning" which is "perfect during training"; this indicates Adam aids in faster convergence. The learning rate was set to 0.00001 as fine tuning was being performed on the pre-trained models, using a larger learning rate would have resulted in unstable training.

[28] As the task is a multi-classification problem, it limits the loss functions which can be used. Log Loss cannot be used due to it being exclusively made for binary classification. Hinge Loss cannot be used due to being used for SVMs rather than CNNs. As a result, Cross-Entropy Loss was chosen.

The batch size of 16 was chosen due to GPU memory constraints on the Otter Lab Machines. When a batch size of 32 was set, training was interrupted due to a lack of memory. To combat this, and speed up training, the num_workers parameter in the data loaders were set to 8. This helps to leverage some level of parallel processing, as the data for each batch can be loaded simultaneously [29].

3.4.3. Architectural alterations:

To alter the pre-trained models to align with the classification of emotions, modifications were made to the classifier/fully connected layers. The models both initially had an output of 1000 classes as they were both trained on IMAGENET, a dataset which contains 1000 classes. As this project only classifies 7 emotions, these layers had to be altered. In the case of MobileNetV2, the initial classifier layer was replaced with a new linear layer which mapped the 1280 in-features from the prior layer to 7 out-features. The alterations to ResNet followed similarly, the fully connected (fc) layer was replaced with a linear layer that mapped the 512 in-features to 7 out-features.

3.4.4. Layer Freezing:

Freezing layers and performing transfer learning allows for the fine-tuning of the weights of the unfrozen layers within the network; the values are updated rather than newly generated. The most common practice involves freezing all convolutional layers until the final classification/full connected layer; allowing for the retention of previously calculated weight values. Retaining these weight values allows the network to perceive and detect features (i.e., edges) without having to be freshly trained and learning these features again, risking overfitting.

Within this section, 2 approaches to freezing layers were explored:

- Approach 1: Freezing until the classifier/full connected layer
- Approach 2: Freezing until the 16th layer (MobileNetV2), and layer 4 (ResNet).

To test whether the transfer learning approaches were effective, the models will be evaluated using their classification accuracy, and metrics (precision, recall and F1 scores); both macro and micro perspectives will be analysed to gauge per-class and overall performance. The accuracy and loss graphs, along with confusion matrices will be reviewed to ensure the models are learning, and whether they are overfitting data.

3.4.5. Transfer Learning results:

3.4.5.1. Approach 1

Approach 1 involves freezing the convolutional layers and leaving the classification/full connected layers unfrozen. This approach attempts to test whether the pre-trained weights within the prior layers are suited for FER.

Table 7 - Shows the results after training each model on a specific dataset, with only the final layer unfrozen

Model	Dataset	Accuracy (%)	Loss	Precision	Recall	F1-Score
MobileNetV2	CK+	64.52	1.2903	0.0922	0.1429	0.1120
	JAFFE	28.57	1.9435	0.2330	0.2857	0.1964
	KDEF	48.30	1.6147	0.4747	0.4930	0.4695
ResNet-18	CK+	64.52	1.1496	0.0922	0.1429	0.1120
	JAFFE	14.29	1.8954	0.2429	0.1429	0.1643
	KDEF	53.74	1.4090	0.5195	0.5374	0.5202
ResNet-34	CK+	65.59	1.1040	0.2360	0.1587	0.1414
	JAFFE	19.05	1.9533	0.1369	0.1905	0.1552
	KDEF	46.26	1.4829	0.4603	0.4626	0.4545

Phenomenon

[Table 7] represents the results after testing the models individually on the selected datasets. CK+ achieved the highest accuracy of all trained models. Even though this metric was high, the corresponding precision, recall and F1- scores were significantly lower. This phenomenon was investigated; the ResNet-34 trained model was used for analysis.

The loss curves in [Figure 3] suggested that the model was learning the data well; such that, the training and validation curves followed similar paths barely diverging during training. The plateauing after the 10th epoch suggested the model had converged and had learnt the data to its full extent. This idea was further reinforced by the accuracy curves in [Figure 2]; both curves rapidly increased, converging between 65%-70%.

Even though these graphs indicated none-minimal overfitting was occurring, both the confusion matrix and scoring metrics did. The confusion matrix in [Figure 4] showed the model was unable to classify any emotions other than neutrality. This was due to the class imbalance within the CK+ dataset; the neutral class is ~66% of the CK+ dataset. This dominance led to the poor generalisation of the other emotions; the model updated more frequently around the neutral class, resulting in the loss of information in respect to the other classes.

Assessing the precision and recall scores [Figure 4] further reinforced the idea of overfitting; for example, the neutral class had a recall score of 1.0, whilst every other emotion had a score near to/or zero. The model failed to correctly predict the less dominant classes due to the lack of features learnt during the training phase. Reviewing the F1-score for the model further highlighted the poor performance across all classes.

As a result of the biased classification performance and overall poor generalisability, the CK+ trained models were disregarded from further analysis.

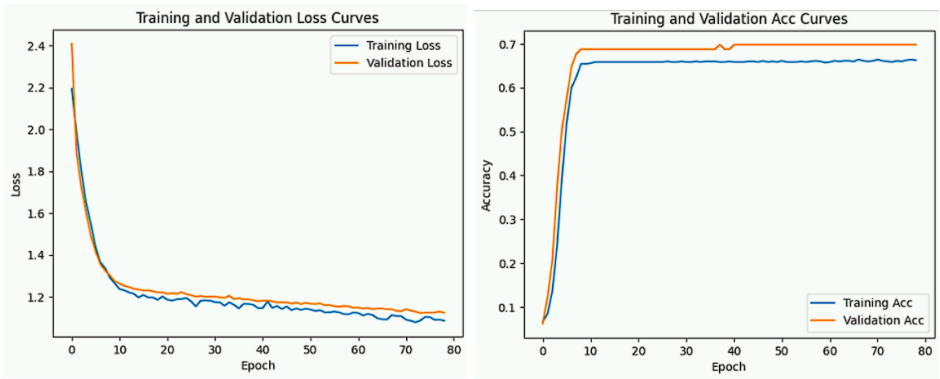
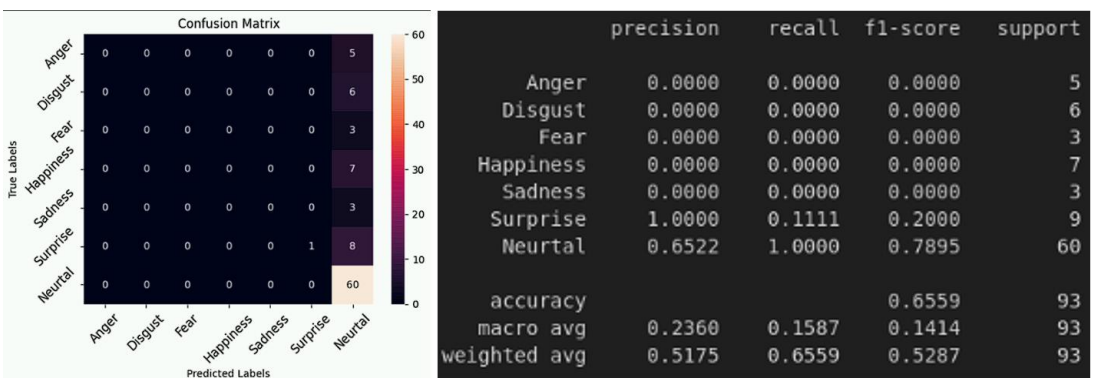


Figure 3 - Shows the loss and accuracy curves of the ResNet-34 architecture, trained on CK+



	precision	recall	f1-score	support
Anger	0.0000	0.0000	0.0000	5
Disgust	0.0000	0.0000	0.0000	6
Fear	0.0000	0.0000	0.0000	3
Happiness	0.0000	0.0000	0.0000	7
Sadness	0.0000	0.0000	0.0000	3
Surprise	1.0000	0.1111	0.2000	9
Neurtal	0.6522	1.0000	0.7895	60
accuracy			0.6559	93
macro avg	0.2360	0.1587	0.1414	93
weighted avg	0.5175	0.6559	0.5287	93

Figure 4 - Shows the confusion matrix and scoring metrics of the ResNet-34 architecture, trained on CK+

Worst Model

The lowest accuracy was achieved by JAFFE trained on ResNet-18; it performed with an accuracy of 14.29%. The model achieved a precision of 0.2429 and a recall of 0.1429, the higher precision score suggests the model was accurate when making the correct prediction to an emotion, even though it rarely predicted correctly. As a result of this, the F1-score remained low at 0.1643.

Reviewing the training and validation loss curves for accuracy showed a great level of divergence and overfitting; for example, the validation loss was consistently higher than training loss, never reaching below 2.10 [Figure 5]. The model

may have been memorising the training samples, rather than generalising. Two factors within the graph showed this: the plateauing validation loss after the 15th epoch and the largely growing gap between both curves.

The accuracy graphs further reinforced the ideas discussed previously. The model had sub-optimal performance; for example, the accuracy curves constantly fluctuated [Figure 5]. The validation curve began to plateau at 0.000 after the 16th epoch. No meaningful features were learnt during training, the model was simply guessing. As the dataset is balanced, it can be concluded these results are due to underfitting as there are not enough samples to learn features from.

The model's confusion is clearly depicted within its confusion matrix [Figure 6]. Some samples are classified correctly across the classes: anger, disgust and fear; whilst the majority are misclassified. The scoring table in [Figure 6] shows the poor recall scores for each class, 4 of the 7 classes have a recall of 0.0000; the lack of feature knowledge has resulted in the model having no ability to realise how emotions are different to one another. Classes like happiness and neutrality which see better results in other studies. have failed to be recognised by the model.

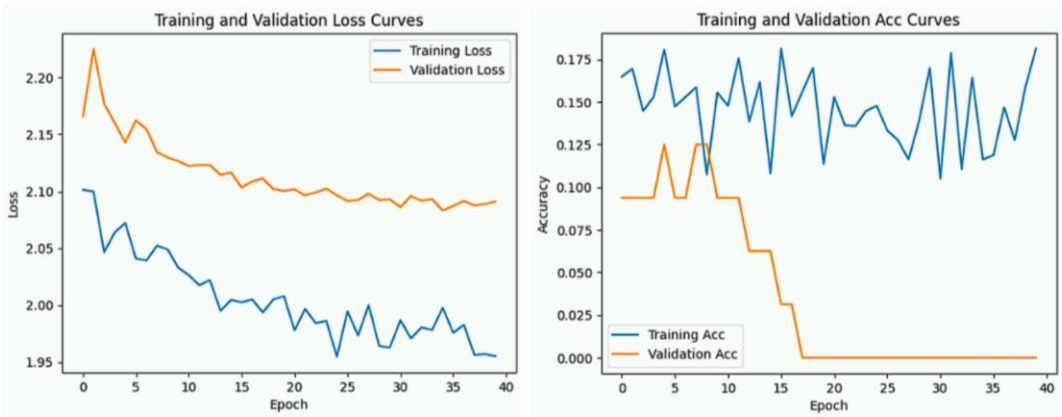


Figure 5 - Shows the loss and accuracy graphs for the ResNet-18 architecture, trained on JAFFE

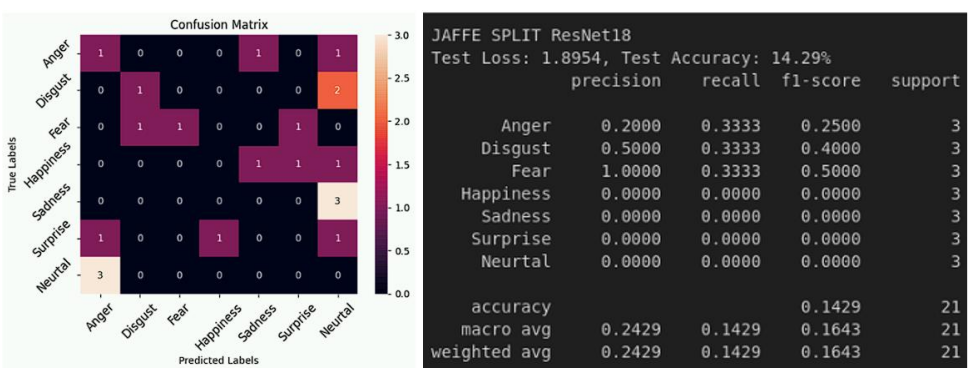


Figure 6 - Shows the confusion matrix and scores metrics for the ResNet-18 architecture, trained on JAFFE

Best Model

The best performing model and dataset combination was KDEF trained on ResNet-18; the model possessed some level of generalisability and classification power. The precision, recall and F1-scores of this model were the highest among others at 0.5195, 0.5374 and 0.5202 respectively. These scores show the model could classify emotions correctly, whilst also understanding which emotions it could not be.

After evaluating the training and validation curves for loss [Figure 7], several observations were made related to the model's ability to learn. For example, both curves followed the same path, with minimal divergence from one another; this indicated that no overfitting was occurring, and that the model was learning. The curve did not exhibit plateauing throughout the training process, suggesting the model had a greater capacity to learn.

The accuracy training and loss curves differ slightly [Figure 7]; such that there is some divergence between the training and validation curves. However, during later epochs, this gap begins to close, indicating the model is beginning to adjust to newly seen data and all it needs is more epochs to better its generalisation ability.

The model was able to classify all emotion classes, with some level of misclassification. Unlike the previous example, all the classes had a value for F1-score, indicating the model learnt some features relating to each class. Most of these misclassifications occur between emotions which shared similar features; for example, anger and disgust [Figure 8]. The model was not able to clearly distinguish differences between these two classes, possibly due to the layer's parameter limitations. The model struggled significantly when attempting to classify fear, often confusing it with surprise; this indicated the model may struggle to identify less dominant features, relying heavily upon those that were clearly presented.

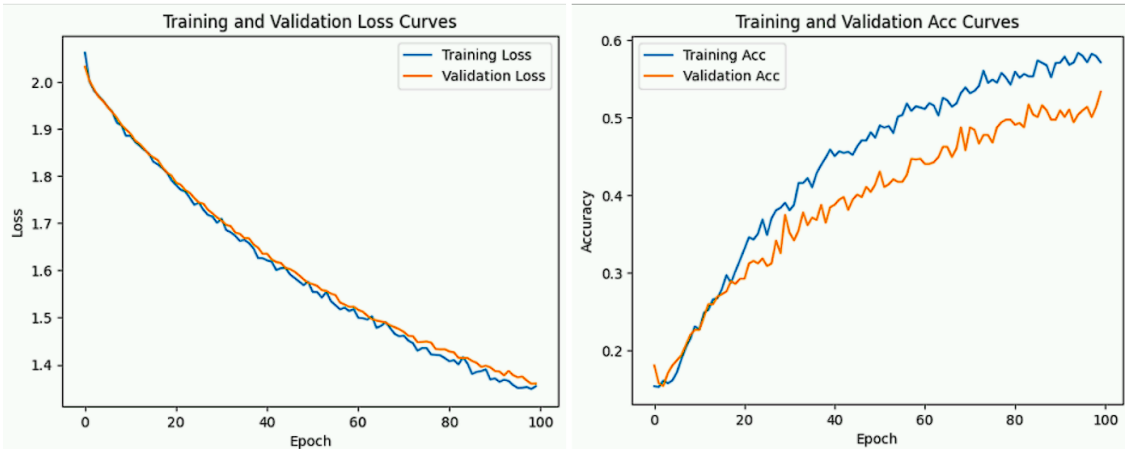
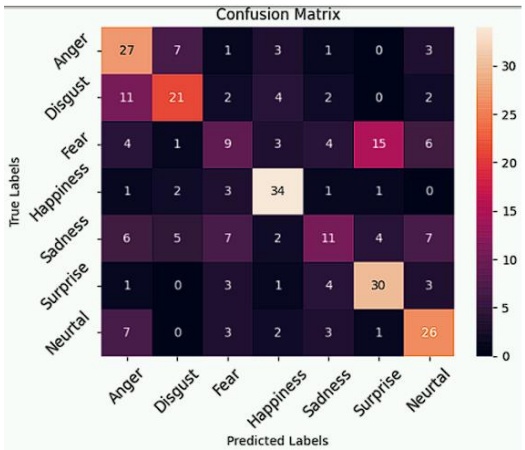


Figure 7 - Shows the loss and accuracy graphs for the ResNet-18 architecture, trained on KDEF



	precision	recall	f1-score	support
Anger	0.4737	0.6429	0.5455	42
Disgust	0.5833	0.5000	0.5385	42
Fear	0.3214	0.2143	0.2571	42
Happiness	0.6939	0.8095	0.7473	42
Sadness	0.4231	0.2619	0.3235	42
Surprise	0.5882	0.7143	0.6452	42
Neurtal	0.5532	0.6190	0.5843	42
accuracy			0.5374	294
macro avg	0.5195	0.5374	0.5202	294
weighted avg	0.5195	0.5374	0.5202	294

Figure 8 - Shows the confusion matrix and scores metrics for the ResNet-18 architecture, trained on KDEF

KDEF had a larger sample size across all splits and was balanced. This came in advantageous when training, as it ensured each class was represented equally and had equal opportunity to perform weight updates. As there were more samples within the classes, it enabled the model to identify and learn the key features related to each emotion.

Conclusion

Approach 1's training demonstrated that there was a limitation to performance with only the last layer unfrozen; for example, in [Table 7] the accuracy metrics were consistently low – especially when the JAFFE dataset was tested. Most of the F1-scores remained below 0.50, with only ResNet-18's KDEF split achieving above that, with 0.5202; this poor performance indicated that the architectures failed to learn defining features for each emotion class and then got confused when an attempt was made to predict them. This confusion and misclassifications were further reinforced when the loss values were reviewed; all loss values ranged between 1-2. This indicated the models were penalising themselves at a greater rate due to the number of misclassifications; meaningful features were not learnt.

In addition, the performance was heavily reliant on the specifications of the dataset; for example, the class distribution and dataset size. If the dataset was heavily skewed towards a certain class (i.e., CK+), the model become biased. The JAFFE results highlight the importance of the dataset size. If there is an inadequate amount of training data, the model will not learn distinguishing features and will arbitrarily guess during classification.

The sub-optimal performance of the models may have been due to a lack of trainable parameters present; for example, only the classifier/full connected layer were unfrozen. The bottleneck forced the model to classify based on the pre-trained features (i.e., edges) with little influence of the newly trained features. As a result of this, the second approach attempts to unfreeze more layers so more parameters can be trained, hopefully capturing better features.

3.4.5.2. Approach 2

After reviewing the performance metrics from Approach 1, which finely-tuned the models with only their final layer unfrozen, it was determined that more layers should be unfrozen to allow the models to learn specific features involved in FER. Unfreezing more layers allows for greater adaptation by introducing a greater number of parameters to be trained. Approach 2 involves unfreezing layers 16-19 are unfrozen for MobileNetV2, and layer 4 is unfrozen for ResNet.

Reviewing [Table 8] shows the improvement of unfreezing more layers; the performance of most of the models improved drastically. The models were correctly classifying samples with less false positives and negatives; this was indicative of the increased F1-scores. In addition, the loss across the results decreased significantly, suggesting training was more stable, and the model was making less misclassification.

Table 8 - Shows the results after training each model on a specific dataset, with more layers unfrozen

Model	Dataset	Accuracy (%)	Loss	Precision	Recall	F1-Score
MobileNetV2	CK+	80.65	0.6018	0.3998	0.4510	0.4195
	JAFFE	28.57	1.9555	0.2330	0.2857	0.1964
	KDEF	78.91	0.5319	0.7887	0.7981	0.7875
ResNet-18	CK+	91.40	0.2914	0.7518	0.7524	0.7503
	JAFFE	80.95	0.4566	0.8095	0.8095	0.7973
	KDEF	90.14	0.2981	0.9033	0.9014	0.9004
ResNet-34	CK+	90.32	0.4025	0.7188	0.7048	0.7058
	JAFFE	80.95	0.3800	0.8595	0.8095	0.7963
	KDEF	87.41	0.3776	0.8740	0.8741	0.8722

Best result

The highest accuracy was achieved by the CK+ dataset trained on ResNet-18. Analysis of the validation and training loss graph in [Figure 9] showed how the model adapted to the new training data. There was a rapid decrease in loss within the first 7 epochs indicating that the model was learning from the data. This point is further reinforced after viewing the curves; they followed the same path with minimal divergence, suggesting overfitting was not occurring at this stage. However, after the 12th epoch, the curves began to diverge from one another; for example, the training curve continued decreasing whilst the validation curve began to plateau. The model has began to memorise.

The confusion matrix [Figure 10] further highlights the increased performance; most classes are correctly classified. Neutral, Happiness and Surprise are classified with near perfect accuracy. The overall high accuracy suggests the model captured relevant features and gained the ability to classify using them. However, some emotions were still misclassified, in particular Sadness. Reviewing the confusion matrix showed that it was never correctly classified and was confused as neutrality on every occasion. This is likely due to the class imbalance mentioned in prior sections. The model was unable to identify features specific to sadness and only related them to neutrality as it was the most dominant class.

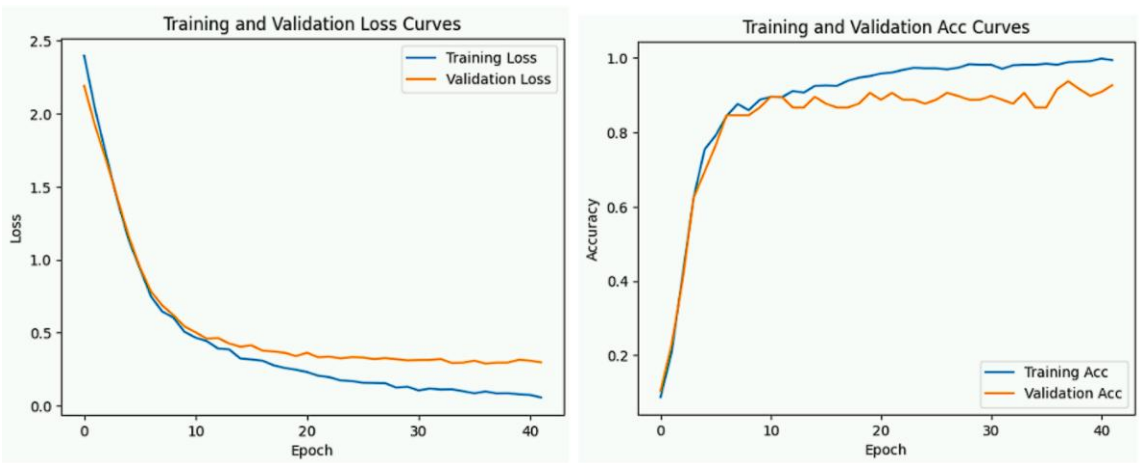
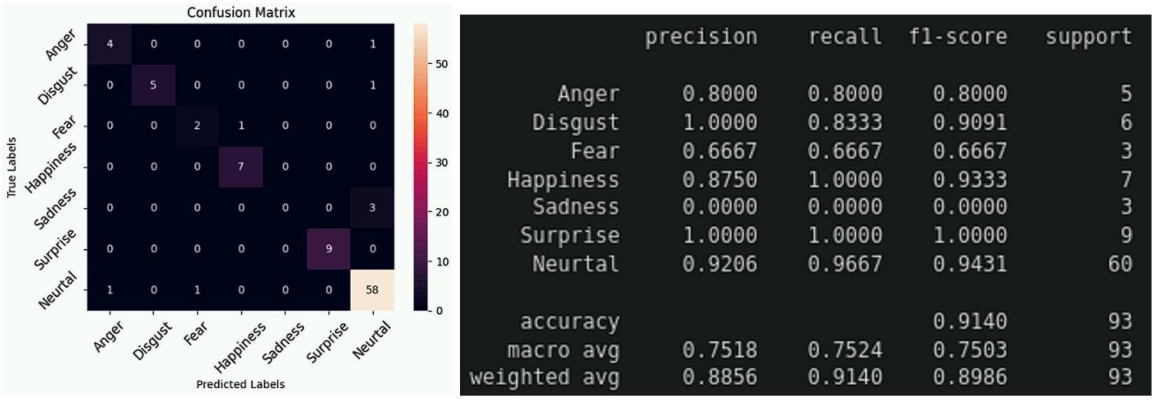


Figure 9 - Shows the loss and accuracy graphs for ResNet-18, trained on CK+



	precision	recall	f1-score	support
Anger	0.8000	0.8000	0.8000	5
Disgust	1.0000	0.8333	0.9091	6
Fear	0.6667	0.6667	0.6667	3
Happiness	0.8750	1.0000	0.9333	7
Sadness	0.0000	0.0000	0.0000	3
Surprise	1.0000	1.0000	1.0000	9
Neurtal	0.9206	0.9667	0.9431	60
accuracy			0.9140	93
macro avg	0.7518	0.7524	0.7503	93
weighted avg	0.8856	0.9140	0.8986	93

Figure 10 - Shows the confusion matrix and score metrics for the ResNet-18 architecture, trained on CK+

Even though the CK+ trained on the ResNet-18 architecture performed the best in terms of accuracy, it was outperformed by KDEF trained on ResNet-18. It achieved an F1-Score of 0.9004 in contrast to 0.7518; this suggested that the KDEF trained model was more generalised, the CK+ model struggled to identify the minority classes due to the lack of data within them.

Worst result

The lowest accuracy was achieved by KDEF on MobileNetV2, it had a classification accuracy of 78.91%. Even though this is the worst performing model in the testing set, it still performed extremely well in contrast to its counterpart in approach 1. Within the first 18 epochs, both training and validation loss curves rapidly decreased, following similar paths [Figure 11]. The lack of divergence indicated the model was learning; the curve was relatively smooth, suggesting there was minimal instability. Both follow a downward trend and begin to plateau after the 40th epoch; however, a slight gap between the curves began to form as training continued. This suggested that the model was overfitting slightly to the training data.

[Figure 11] showed both accuracy curves following similar paths, with minimal divergence. There was a sharp increase in accuracy until the 18th epoch, further reinforcing that the model was learning well. Again, after the 40th epoch there was slight divergence.

The confusion matrix in [Figure 12] showed the strength of the model's classification performance; happiness was classified with the highest accuracy, only having one misclassification. Its F1-score of 0.9425 indicated the model was able to clearly distinguish the differences between it and the other classes. Other emotions such as Surprise, Neutral, and Disgust also had strong classification accuracies, however the model struggled to predict them correctly, due to their lower F1-scores (they were misclassified more).

The model struggled to classify emotions which were presented similarly, symmetry within the confusion matrix especially within the disgust and anger classes shows this. The emotions present similarly; the model confuses these shared features and classifies them as one another. This results in an overall drop in classification accuracy and F1-Score.

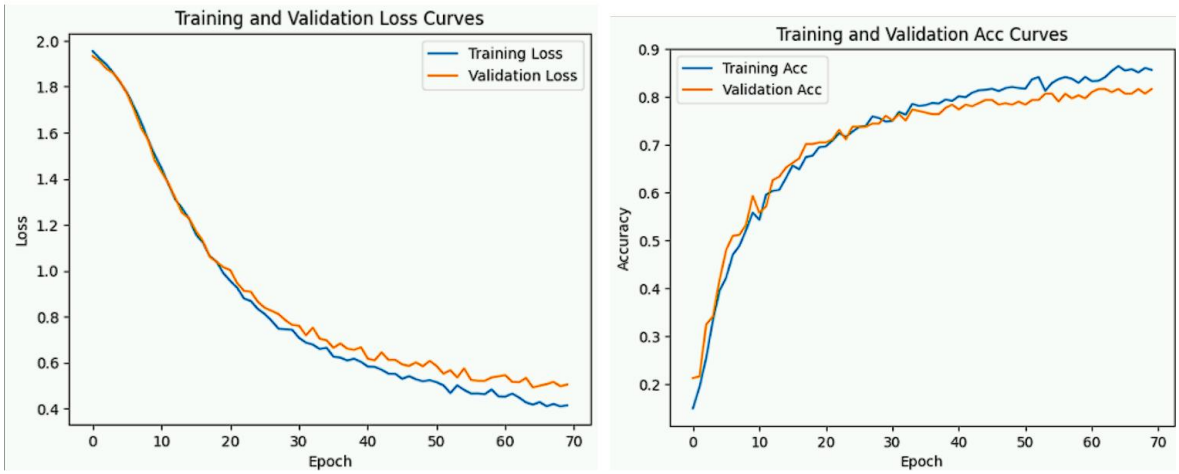
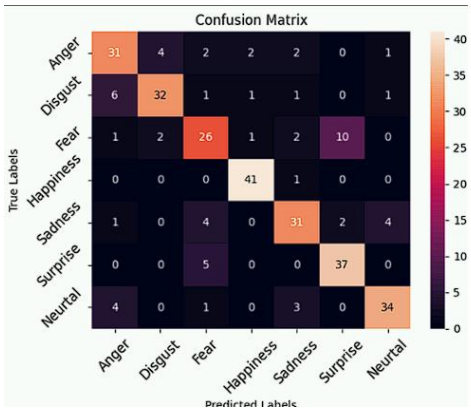


Figure 11 - Shows the loss and accuracy graphs for MobileNetV2, trained on KDEF



	precision	recall	f1-score	support
Anger	0.7209	0.7381	0.7294	42
Disgust	0.8421	0.7619	0.8000	42
Fear	0.6667	0.6190	0.6420	42
Happiness	0.9111	0.9762	0.9425	42
Sadness	0.7750	0.7381	0.7561	42
Surprise	0.7551	0.8810	0.8132	42
Neurtal	0.8500	0.8095	0.8293	42
accuracy			0.7891	294
macro avg	0.7887	0.7891	0.7875	294
weighted avg	0.7887	0.7891	0.7875	294

Figure 12 - Shows the confusion matrix and score metrics for MobileNetV2, trained on KDEF

Further Evaluation

Even though the metric in the results table indicates there are no inherent issues when using the JAFFE dataset for training, its loss graphs show a key problem. Reviewing the loss curves shows an excessive amount of overfitting that occurs when the dataset is used to train the ResNet models. This occurs due to the small size of the dataset; the model is unable to identify generalisable features and begins to memorise the data.

Best Transfer Learning model

After assessing the performance of all models across every dataset, there was an apparent trend. The ResNet-18 architecture achieved the best performance in terms of accuracy, with comparable precision recall and F1 metrics. The training was also seen as the most stable, such that the loss values are some of the lowest within the [Table 8].

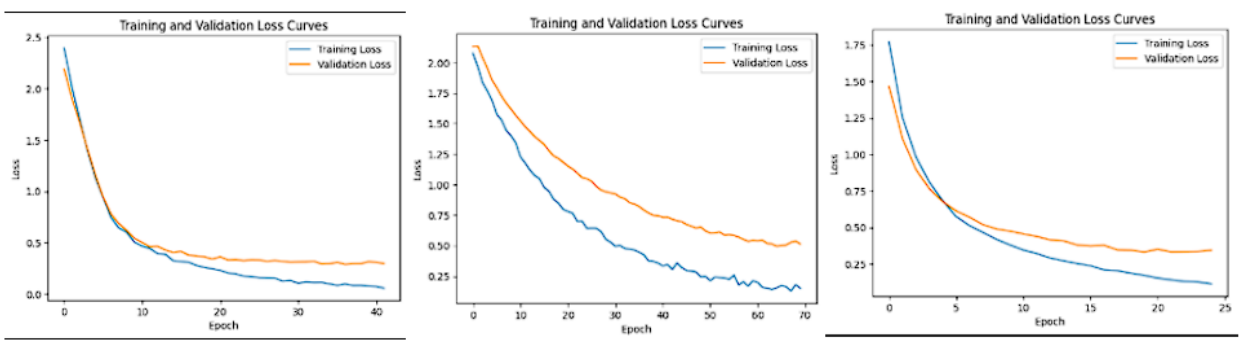


Figure 13 - Shows the loss graphs using the ResNet-18 architecture in order of CK+, JAFFE, and KDEF

Whilst the model performs with the highest classification accuracy, there was some indication of the training data being memorised. The loss curves in [Figure 13] all exhibited some level of overfitting, especially within JAFFE. However, overfitting is not a rare occurrence when utilising the ResNet architecture. As the architecture was built for image classification using larger datasets (IMAGENETV2), its depth was increased. With the introduction of more layers (i.e., convolutions, batch normalisation, ReLU), there was an increase in the number of trainable parameters; these additional parameters could be used to learn deeper features that prior layers did not identify. As the FER datasets are relatively small, the deeper layers within the model begin to memorise the features instead of generalising them. Hence when new data is presented, the architecture struggles to classify correctly as it has not understood the underlying features; this causes the validation curve to diverge from the training curve.

The performance of ResNet-34 was also comparable to ResNet-18; the architecture achieves similar results but with worsened precision, recall and F1 metrics. The reduced performance was most likely due to the lack of stability during

training, such that the loss values were larger than ResNet-18's, indicating the model was getting confused during training. These metrics may also be worse due to the increased depth of the architecture; the deeper layers may begin memorising noise over meaningful features.

Cross-set generalisability

To assess the generalisability of each model, they were tested against the other datasets which they were not trained on. For example, the ResNet-18 model trained on KDEF was tested against CK+ and JAFFE. Performing this test allowed for the models raw classification power to be tested; altering the domain of data hoped to identify whether the models were biased towards the trained dataset, or whether underlying features have been learnt.

[Table 9] shows the accuracies of the datasets tested. The overall results were extremely poor, barely achieving above 35% in most cases. Even though these models performed well in their respective datasets, their ability to generalise features in a new environment was severely limited. As mentioned in [Dataset Selection Section], the individual datasets were not diverse, such that they only contain images of those from a singular ethnicity in most cases.

Table 9 - Shows the accuracies of the trained models, after attempting to classify data from another domain

Architecture	Trained Dataset	Tested Dataset	Accuracy (%)	F1-Score
MobileNetV2	CK+	JAFFE	14.55	0.0443
		KDEF	14.33	0.1433
	JAFFE	CK+	7.87	0.0787
		KDEF	14.43	0.1088
	KDEF	CK+	22.28	0.2278
		JAFFE	39.44	0.4007
ResNet-18	CK+	JAFFE	22.07	0.1699
		KDEF	30.33	0.2279
	JAFFE	CK+	10.64	0.1036
		KDEF	38.87	0.3525
	KDEF	CK+	42.68	0.3136
		JAFFE	39.91	0.3601
ResNet-34	CK+	JAFFE	32.86	0.2838
		KDEF	31.18	0.2718
	JAFFE	CK+	15.41	0.1740
		KDEF	30.57	0.2707
	KDEF	CK+	27.27	0.3030
		JAFFE	36.62	0.3380

The architectures trained on the KDEF dataset produced the best results in terms of F1-score; even though accuracy was generally poor, the architecture had some level of generalisability. This was most likely due to the size and distribution of the dataset; each class was given an equal opportunity to be represented. Whilst on the other hand, architectures trained using the JAFFE dataset struggled significantly to generalise; MobileNetV2 trained on JAFFE achieved the lowest F1-score (0.0787) when tested against the CK+ dataset. Comparing the architectures that showed ResNet-18 outperformed both ResNet-34 and MobileNetV2; it achieved F1-scores that differed marginally in range. The architecture was able to balance its depth and feature extracting capabilities; generalisable features were learnt in addition to dataset specific ones.

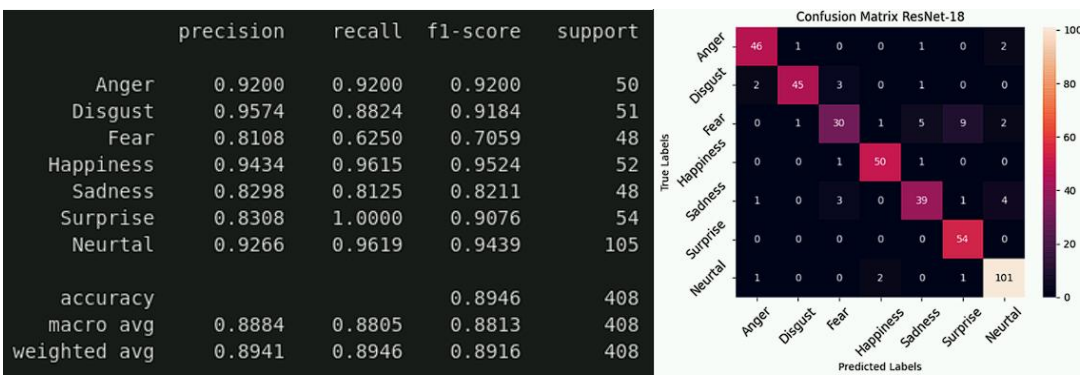
Compiled Dataset Test

However, to produce a model that has a greater capability to generalise a choice was made to compile the datasets. This would help address the domain adaptation issue; the training data would be more diverse and have greater variance, such that meaningful features could be learnt, instead of those which were dataset specific.

Table 10 - Shows the accuracies of the models after being trained on the compiled dataset

Architecture	Accuracy (%)	Loss	Precision	Recall	F1-Score
MobileNetV2	82.35	0.4817	0.8127	0.8080	0.8059
ResNet-18	89.46	0.3288	0.8884	0.8805	0.8813
ResNet-34	86.52	0.4345	0.8580	0.8453	0.8472

ResNet-18's strengths were further exemplified after being trained using the compiled dataset; it achieved the highest accuracy at 89.46% with a loss of 0.3288. In addition, ResNet-18 also boasted the highest scores in terms of precision, recall and F1. [Figure 14] shows the per-class metrics, the architecture achieved strong performance across most of the class; however, it struggled slightly when attempting to classify fear. It had the most misclassification, most commonly getting confused as surprise and sadness. Even though overfitting was present [Figure 14], it remained at a minimum; the reasoning for its occurrence was most likely due to the increased depth of the architecture, and size of the dataset.



	precision	recall	f1-score	support
Anger	0.9200	0.9200	0.9200	50
Disgust	0.9574	0.8824	0.9184	51
Fear	0.8108	0.6250	0.7059	48
Happiness	0.9434	0.9615	0.9524	52
Sadness	0.8298	0.8125	0.8211	48
Surprise	0.8308	1.0000	0.9076	54
Neurtal	0.9266	0.9619	0.9439	105
accuracy			0.8946	408
macro avg	0.8884	0.8805	0.8813	408
weighted avg	0.8941	0.8946	0.8916	408

Figure 14 - Shows the per-class metrics of ResNet-18, trained on the compiled dataset

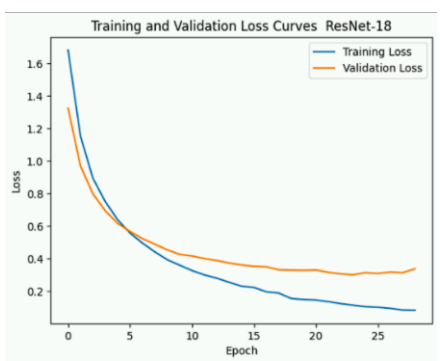


Figure 15 - Shows the loss graph for training ResNet-18 using the compiled dataset

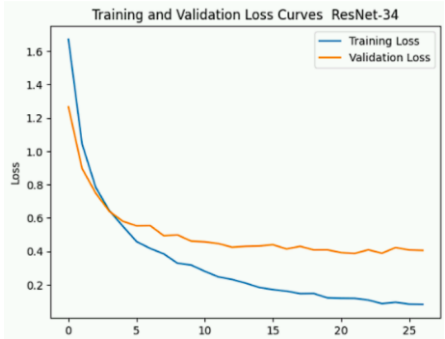


Figure 16 - Shows the loss graph for ResNet-34 trained on the compiled dataset

Although the same cannot be said for ResNet-34; the training and validation curves in [Figure 16] began to diverge from the 4th epoch, this gap continued to grow, surpassing that shown in [Figure 15]. Even though ResNet-34 had more layers and should have technically allowed for more features maps to learnt, the size of the dataset prevented it. The architecture began to memorise the training data. As a result, the precision, recall and accuracy metrics are worse than ResNet-18.

MobileNetV2 performed well, but achieved the lowest accuracy across all tested models. In addition to its lesser accuracy, it also performs worse across metrics such as precision, recall and loss. Reviewing the metrics [Figure 17] revealed that MobileNetV2 struggled with classifying emotions which presented similarly. Even though the loss graphs indicated the architecture was learning, it may not have been able to capture all relevant features due to its limited capacity. [Figure 18] shows the curves beginning to diverge after the 20th epoch; the validation curve plateaus from the 50th epoch, whilst the training curve continues decreasing. As MobileNetV2 is a lighter weight architecture, it has less trainable parameters; ResNet has approximately 5x as many trainable parameters. The lack of these parameters could result in deeper more complex features not being represented to their fullest extent; prioritisation could be made to more distinguishable features.

	precision	recall	f1-score	support
Anger	0.6949	0.8200	0.7523	50
Disgust	0.8400	0.8235	0.8317	51
Fear	0.7647	0.5417	0.6341	48
Happiness	0.8947	0.9808	0.9358	52
Sadness	0.7857	0.6875	0.7333	48
Surprise	0.8305	0.9074	0.8673	54
Neurtal	0.8785	0.8952	0.8868	105
accuracy			0.8235	408
macro avg	0.8127	0.8080	0.8059	408
weighted avg	0.8226	0.8235	0.8193	408

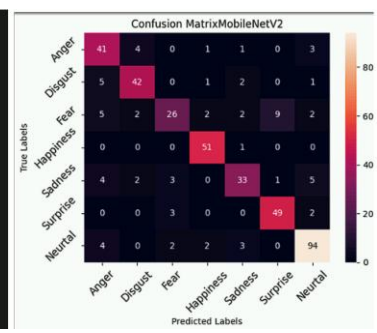


Figure 17 - Shows the per-class scores for MobileNetV2 trained on the compiled dataset

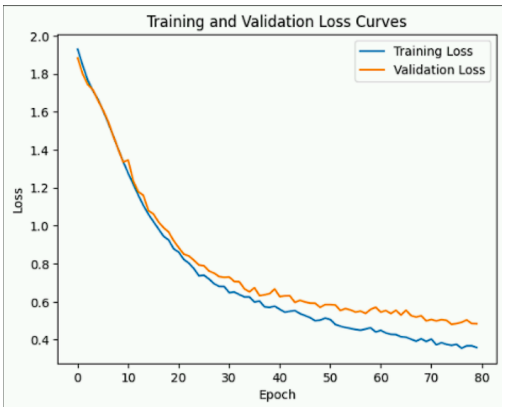


Figure 18 - Shows the loss graph for MobileNetV2 trained with the compiled dataset

Conclusion:

This section investigated the effects of transfer learning for performing FER: MobileNetV2, ResNet-18 and ResNet-34 were the tested architectures. Two approaches to transfer learning were taken: unfreezing only the fully connected/classifier layer and unfreezing further convolutional layers (layer 4 in ResNet, and layers 16-19 in MobileNetV2). These two approaches were tested to see whether the pre-trained architectures had the ability to use previous training information to classify emotions.

Results showed that training the architectures with only their fully connected/classifier layers unfrozen led to suboptimal performance. The architectures trained stably such that their loss graphs did not exhibit extreme levels of overfitting; however, their confusion matrices and score metrics showed otherwise. Unfreezing more layers further improved the overall performance of each architecture, classification accuracy and scoring metrics increased; the individual architectures began to learn features present in the data. Even though performance increased, overfitting was introduced; as there were more trainable parameters, the architectures began to learn noise within the data rather than its generalisable features.

Despite the overall increase in performance after unfreezing more layers, several drawbacks became apparent – especially when the testing domain was shifted. The architectures struggled significantly when attempting to classify images from another dataset, barely achieving over 35% in most cases. These findings resulted in the datasets being compiled; this action hoped to provide a larger samples size with greater variance such that a more robust architecture could be trained.

This final approach led to better results overall, in terms of raw classification performance and generalisability; most classes were correctly predicted. Classes such as fear were quite commonly mis-classified, getting confused with surprise and anger; however, even with these misclassifications, the overall scoring metric remained strong. The ResNet architectures showed low-moderate levels of overfitting; most likely due to their increase depth and parameter count.

Transfer learning is effective when attempting to perform FER, but its performance is heavily reliant on the structure of the architecture and dataset used. The custom architecture section hopes to further improve performance, developing a shallower architecture which still boasts strong performance even on less dominant classes.

3.5. Custom Architecture:

Even though transfer learning proved effective at performing FER, it was limited by several factors; most notably being the susceptibility to overfitting. The architectures were relatively deep, utilising upwards of 33 convolutional layers. Even

though a greater number of feature maps were produced, which should have aided in the identification of new features, it rarely did. The architecture began to memorise dataset specific features (i.e., background lighting or general noise), instead of generalisable features used to identify emotions.

As result of this, a custom architecture was proposed; it was specifically designed for FER, deriving its weight values from FER datasets rather than object recognition ones. Due to the susceptibility of overfitting, the architectures designed were shallower than their pre-trained counterparts. Their creation took an iterative approach; a simple baseline architecture was first created, utilising only one convolutional layer. This baseline was then tested and reviewed and further optimised. This was achieved with the introduction of more convolutional layers for feature extraction; reduction in learning rate to ensure steady convergence; or the addition of a learning rate schedulers to incrementally reduce the learning rate throughout training.

To further aid in the optimisation of the architectures, a combination of loss function and optimiser pairings were tested. Section 3.1.5 highlighted the challenges of using imbalanced datasets; hence two loss functions were tested – specifically Cross Entropy Loss and Weighted Cross Entropy Loss. With both loss functions, it was decided to test the Adam and SGD optimisers; Adam was chosen due to its strong performance within [27], and SGD was chosen due to its usage within GIMEFIVE [5].

3.5.1. Testing Environment:

Similarly to section 3.4.1, the same testing environment will be utilised.

3.5.2. Architectural Variants:

Four architectures were tested within this section, each one growing the baseline architecture in complexity and parameter count. Each stage introduced more layers, and further hyper-parameter optimisation. The training parameters which remained constant throughout this process are listed below:

- Epochs: 50
- Batch Size: 16, num_workers = 8

There were two configurations running simultaneously whilst training the model; each configuration utilised a different optimiser and loss function pairing. The loss functions and optimisers that were used are as follows:

- Optimisers:
    - Adam
    - SGD
- Loss Function:
    - Cross Entropy Loss
    - Weighted Cross Entropy Loss

The hyper-parameters for each optimiser will be iterated upon after each new architecture, these parameters include learning rate, weight decay and patience.

3.5.2.1. Architecture 1 – Baseline:

Due to the moderate-large amount of overfitting seen within the transfer learning section, it was proposed that a shallower architecture should be utilised. As a result, the baseline architecture in [Figure 19] was created; its purpose was to test whether a simple CNN had the ability to extract generalisable features, rather than memorising the data.

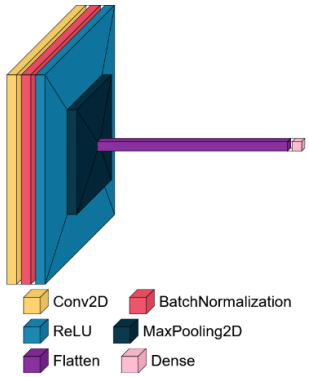


Figure 19 - Shows Architecture 1 structure

Table 11 - Shows the results of Architecture 1

ID	Optimiser	Loss Function	Accuracy (%)	Loss
1	Adam	Cross Entropy Loss	74.02	1.1134
2	SGD	Cross Entropy Loss	74.02	0.7993
3	Adam	Weighted Cross Entropy Loss	73.04	1.1629
4	SGD	Weighted Cross Entropy Loss	75.74	0.7690

[Table 11] shows the strength of using the SGD optimiser; in both weighted and unweighted scenarios it outperformed Adam, achieving a smaller loss value. Even though these metrics showed some indication of classification power, an analysis of the training/validation curves in [Figure 20] showed otherwise.

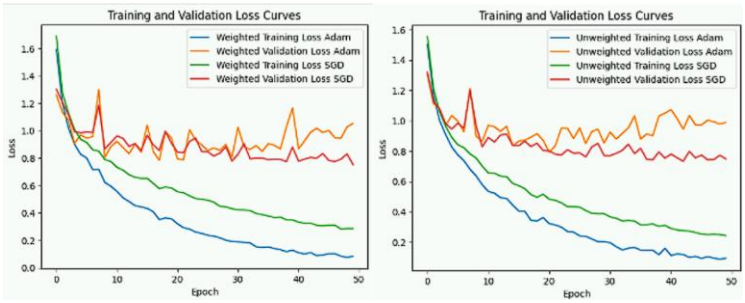


Figure 20 - Shows the loss graphs of Architecture 1

Despite Adam showing early signs of learning, with an initial rapid reduction in loss, signs of overfitting were exhibited in both unweighted and weighted scenarios [Figure 20]. In particular, the weighted variant presented signs of overfitting after the 4th epoch; validation loss began to plateau whilst training loss continued to steadily decrease. Even though Adam converged at a faster rate than SGD, it done so with a greater level of instability; the validation curve exhibited fluctuations between the 8th-38th epoch until the curve began to increase. SGD followed a similar pattern but was far stabler than Adam; such that there was a smaller gap between the training and validation curves.

Reviewing these graphs from a macro perspective showed the drawbacks of utilising the shallower architecture; for example, there was always some level of divergence between both training and validation curves. The architecture was most likely too simple; it was unable to capture key underlying features and rather memorised the training data. In

addition, no regularisation techniques were applied either; their addition could have reduced divergence, and helped create a more adaptable architecture.

3.5.2.1. Architecture 2:

Iterating upon the previous architecture, using the results from its testing, a new architecture was developed. It included an additional convolutional layer and introduced regularisation. The convolutional layer hoped to further develop the architectures' ability to identify features within the data; deeper and less prominent features could be extracted and learnt. Regularisation in the form of dropout was also added to the aid in the reduction of overfitting which was apparent in both [Figure 20] within the prior section. [Figure 21] shows the architecture.

Due to the slower rate of convergence for the SGD optimiser, its learning rate parameter increased from 0.00001 -> 0.0001 in hopes of fastening convergence. Adam's learning rate was also reduced from 0.00001 -> 0.000005, in hopes to reduce the effects of overfitting, and better convergence.

As the validation curves in [Figure 20] exhibited signs of plateauing, a decision was made to implement a learning rate scheduler during the training process. The newly created architecture was first trained without using the scheduler; the loss graphs for the unweighted and weighted trained architectures were then reviewed. Patience was assigned its value after viewing where the curves began to diverge; decreasing the learning rate at this point hoped to ensure convergence could be achieved. The results of testing are displayed within [Table 12].

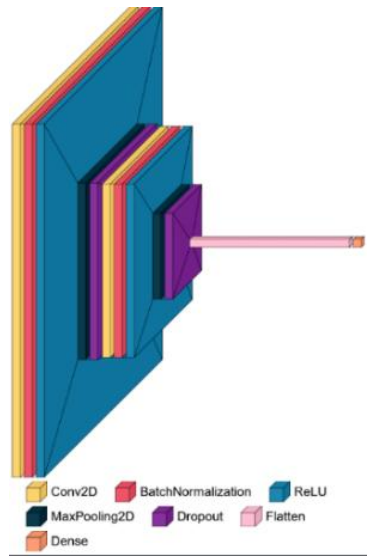


Figure 21 - Shows Architecture 2 structure

Table 12 - Shows the results of Architecture 2

Optimiser	Loss Function	Accuracy (%)	Loss	F1-Score
Adam	Cross Entropy Loss	71.32	0.8370	0.6910
SGD	Cross Entropy Loss	70.34	0.8587	0.6751
Adam	Weighted Cross Entropy Loss	71.57	0.8309	0.7006
SGD	Weighted Cross Entropy Loss	71.57	0.9221	0.6896

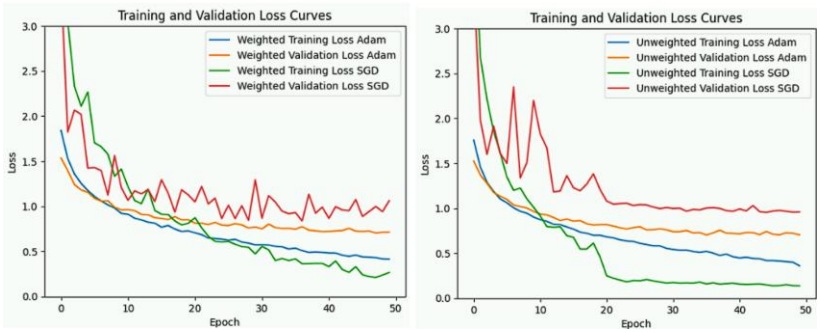


Figure 22 - Shows the loss graphs for Architecture 2

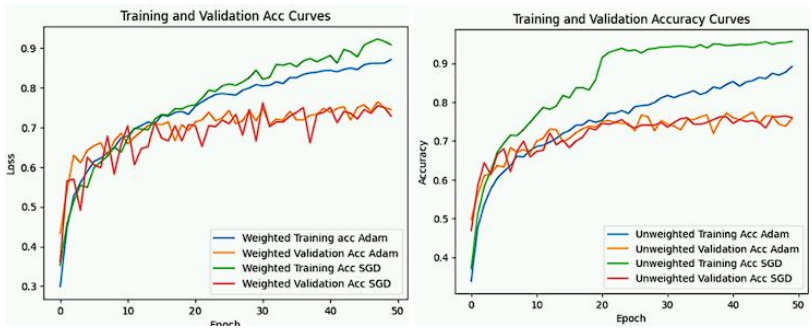


Figure 23 - Shows the accuracy graphs for Architecture 2

[Figure 22] highlights the performance increase after introducing additional layers (i.e., convolutional and dropout) and a learning rate scheduler, especially for the Adam optimiser. In both weighted and unweighted configurations, the training and validation curves followed similar paths, both steadily decreasing; divergence remained minimal, only occurring after the 10th epoch. In contrast, the SGD optimiser fared slightly worse after having its learning rate increased; [Figure 22] and [Figure 23] highlighted the number of fluctuations which occurred during the training process especially within the weighted configuration, suggesting some level of instability. Divergence saw an overall decrease in comparison to Architecture 1, but it was still present. The unweighted configuration clearly represents this; after the 20th epoch the curves have diverged significantly.

```
Unweighted Adam Test Loss: 0.8370, Unweighted Adam Test Accuracy: 71.32%
              precision    recall  f1-score   support

       Anger     0.5424    0.6400    0.5872        50
     Disgust     0.7727    0.6667    0.7158        51
        Fear     0.5484    0.3542    0.4304        48
   Happiness     0.9600    0.9231    0.9412        52
     Sadness     0.5000    0.6667    0.5714        48
    Surprise     0.6970    0.8519    0.7667        54
     Neurtal     0.8723    0.7810    0.8241       105

    accuracy                         0.7132       408
   macro avg     0.6990    0.6976    0.6910       408
weighted avg     0.7255    0.7132    0.7128       408
```

```
Unweighted SGD Test Loss: 0.8587, Unweighted SGD Test Accuracy: 70.34%
              precision    recall  f1-score   support

       Anger     0.6562    0.4200    0.5122        50
     Disgust     0.7727    0.6667    0.7158        51
        Fear     0.5000    0.5417    0.5200        48
   Happiness     0.8654    0.8654    0.8654        52
     Sadness     0.5333    0.5000    0.5161        48
    Surprise     0.7818    0.7963    0.7890        54
     Neurtal     0.7344    0.8952    0.8069       105

    accuracy                         0.7034       408
   macro avg     0.6920    0.6693    0.6751       408
weighted avg     0.7013    0.7034    0.6965       408
```

Figure 24 - Shows the per class metrics for the unweighted loss function, trained with both Adam and SGD

```
Weighted Adam Test Loss: 0.8309, Weighted Adam Test Accuracy: 71.57%
              precision    recall  f1-score   support

       Anger     0.6667    0.6400    0.6531        50
     Disgust     0.7959    0.7647    0.7800        51
        Fear     0.3710    0.4792    0.4182        48
   Happiness     0.9400    0.9038    0.9216        52
     Sadness     0.8065    0.5208    0.6329        48
    Surprise     0.7447    0.6481    0.6931        54
     Neurtal     0.7521    0.8667    0.8053       105

    accuracy                         0.7157       408
   macro avg     0.7253    0.6891    0.7006       408
weighted avg     0.7316    0.7157    0.7176       408
```

```
Weighted SGD Test Loss: 0.9221, Weighted SGD Test Accuracy: 71.57%
              precision    recall  f1-score   support

       Anger     0.7297    0.5400    0.6207        50
     Disgust     0.7255    0.7255    0.7255        51
        Fear     0.4182    0.4792    0.4466        48
   Happiness     0.8727    0.9231    0.8972        52
     Sadness     0.5814    0.5208    0.5495        48
    Surprise     0.7800    0.7222    0.7500        54
     Neurtal     0.7949    0.8857    0.8378       105

    accuracy                         0.7157       408
   macro avg     0.7003    0.6852    0.6896       408
weighted avg     0.7167    0.7157    0.7132       408
```

Figure 25 - Shows the per class metrics for the weighted loss function, trained with both Adam and SGD

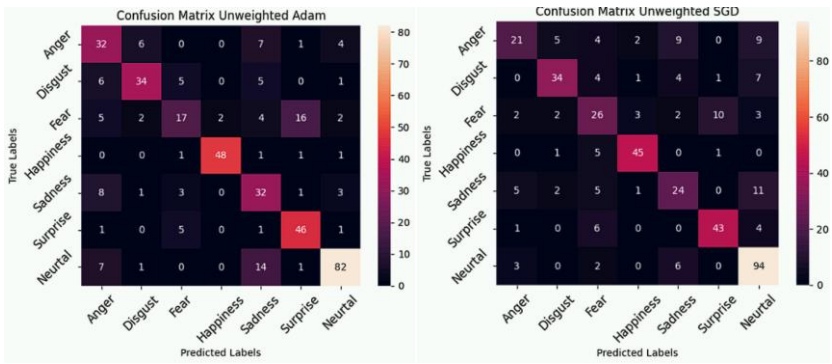


Figure 26 - Shows the confusion matrices for unweighted Adam and SGD

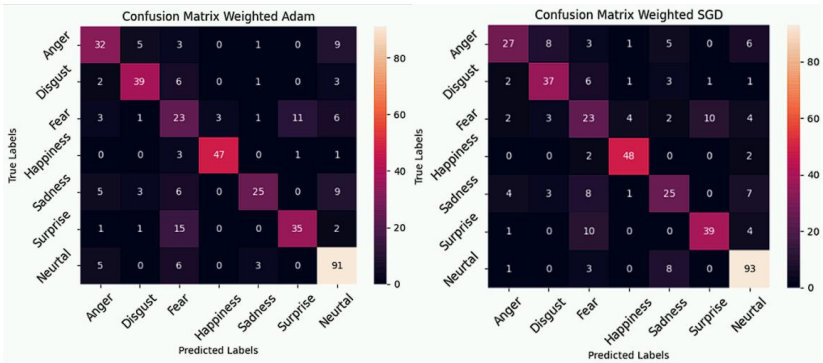


Figure 27 - Shows the confusion matrices for weighted Adam and SGD

Comparing the performance of the loss functions shows weighted loss having an advantage over its counterpart; it exhibited an improvement on generalisation due to the increased F1-Scores seen in [Table 12]. This was true for both optimisers, with Adam seeing an improvement from 0.6910 -> 0.7006, and SGD from 0.6751 -> 0.6896. Emotions such as disgust saw an overall improvement in its F1-Score, increasing from 0.7158 to 0.7800 in Adam; and 0.7158 to 0.7255 in SGD. However, there was a reduction in performance for some classes, particularly surprise; [Figure 24] and [Figure 25] depict its F1-scores decreasing in both configurations. The confusion matrices in [Figure 26] and [Figure 27] show surprise getting predicted as fear, approximately 3x as much in Adam; and 2x in SGD. The weighted loss function caused confusion between emotions which share similar features; this was also seen in the transfer learning section.

The metrics from this section further highlighted the strengths of the Adam optimiser due to it consistently outperforming SGD in terms of stability and overall generalisation. The weighted loss function aided in the classes being represented equally during the training process; minority classes such as fear and anger saw an overall improvement in terms of classification accuracy and scoring metrics. However, its addition negatively impacted the classification of surprise, a minority class.

3.5.2.1. Architecture 3:

Even though architecture 2 improved significantly over architecture 1, it still had its own set of drawbacks; for example, overfitting. Even after the addition of a dropout layer, the model was still memorising the training dataset; [Figure 22] shows the divergence between curves. As well as that, individual classes such as fear were not receiving enough representation; the model struggled to identify and classify images presenting the emotion. As a result, a third iteration was developed [Figure 28]; it introduced an additional convolutional layer, and an additional dropout layer.

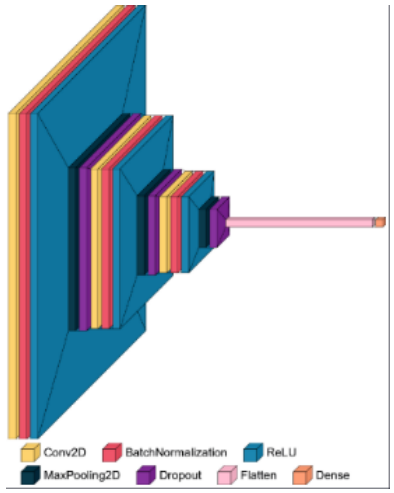


Figure 28 - Shows Architecture 3

To strengthen the effects of regularisation, the existing dropout layers had their threshold probability increased from 0.3 -> 0.4. Even though more neurons would be dropped, there would be a lesser chance of overfitting occurring, and negatively impact the training of the architecture. In addition, the Adam optimisers weight decay parameter increased from 0.001 -> 0.5. Momentum was added to the SGD optimiser, set at 0.93, to help guide the optimiser and converge at a faster rate. The learning rate of the Adam optimiser increased from 0.000005 -> 0.00005; when the architecture was tested using the lower learning rate, stagnation occurred.

In hopes of improving classification and reducing overfitting, a secondary fully connected layer was added. The layer hoped to reduce the number of parameters between the original fully connected layer to the output layer.

Table 13 - Shows the results of training Architecture 3

Optimiser	Loss Function	Accuracy (%)	Loss	F1-Score
Adam	Cross Entropy Loss	77.45	0.6866	0.7418
SGD	Cross Entropy Loss	79.66	0.6454	0.7762
Adam	Weighted Cross Entropy Loss	67.89	0.8431	0.6656
SGD	Weighted Cross Entropy Loss	72.06	0.8676	0.6896

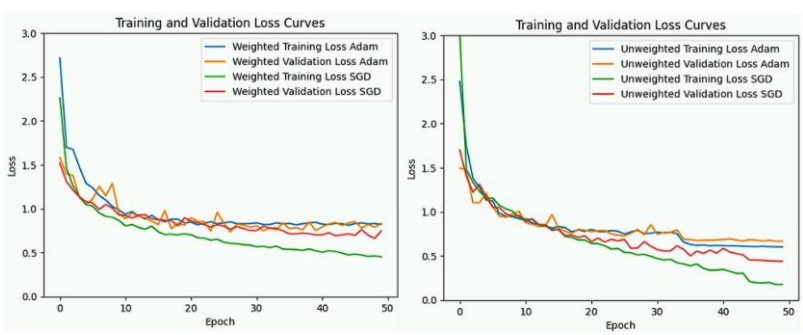


Figure 29 - Shows the loss graphs for Architecture 3

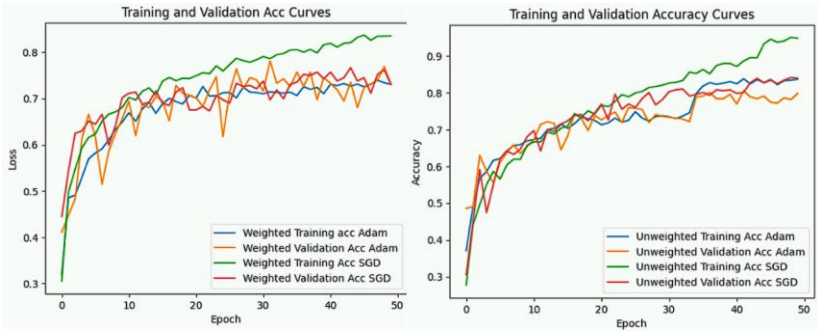


Figure 30 - Shows the accuracy graphs for Architecture 3

Even though the Adam optimiser showed rapid initial decreases during the earlier epochs, the rate of decrease began to reduce as training continued; this was exhibited in both weighted and unweighted scenarios shown in [Figure 29]. Comparing the loss graphs in [Figure 29] to [Figure 22] shows the overall performance increase of the updated architecture. Despite the clear performance advantages in the prior sections, the updated architecture favoured SDG, particularly the unweighted variant. An accuracy of 79.66% was achieved using it; which was the highest among the samples in [Table 13]. The updated optimiser enabled the architecture to converge similarly to Adam, achieving a smaller loss of 0.6454, with some level of instability. Even though a smaller loss was achieved, there was some indication of overfitting; especially in weighted configuration [Figure 29]. Divergence occurred earlier on in training, approximately after the 8th epoch. This idea is further reinforced by the weighted training accuracy curve displayed in [Figure 30], the training curve continues to increase, whilst the validation curve plateaued around 70%.

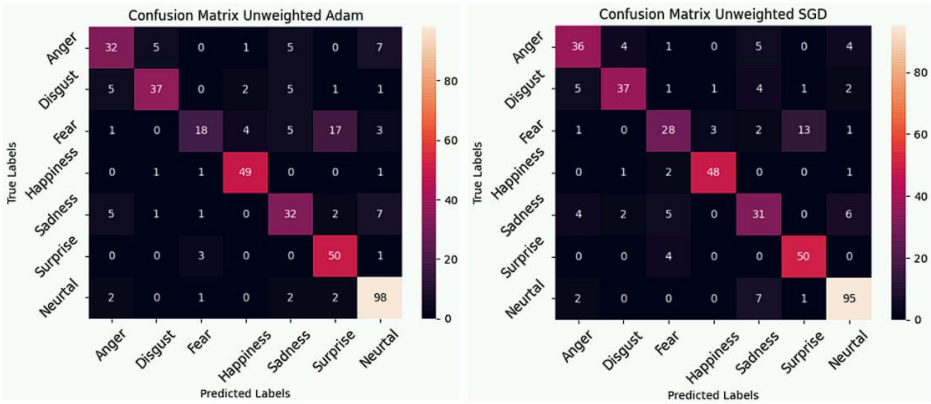


Figure 31 - Shows the confusion matrices for unweighted Adam and SGD

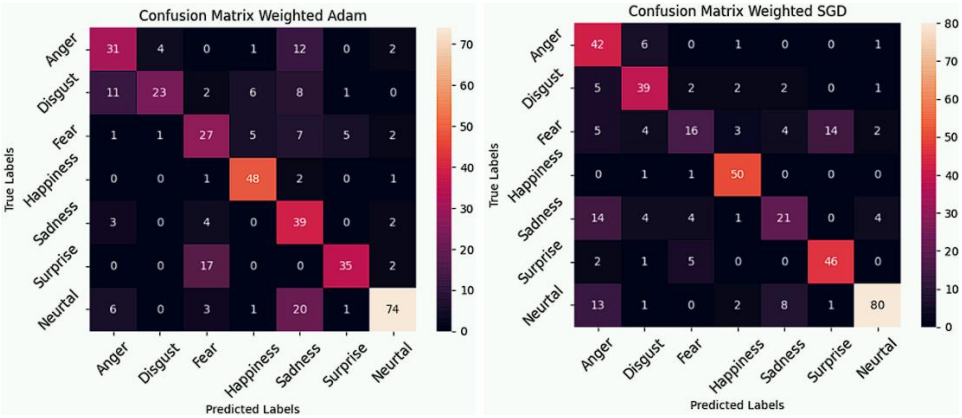


Figure 32 - Shows the confusion matrices for weighted Adam and SGD

Unlike the previous section, the weighted loss function was not advantageous; it performed worse than its unweighted counterpart. The weighted architectures generalisation capabilities were hindered, such that the F1-score decreased by approximately 10%. This performance downgrade can be seen after comparing the confusion matrices in [Figure 31] and [Figure 32].

The unweighted matrices in [Figure 31] show some misclassifications with strong diagonalisation, indicating the architectures were learning features corresponding to each class with some confusion. The best performing classes within the unweighted configuration were happiness, surprise and neutral; these results are most likely due to the classes having the most images within them. Once again, the architecture struggled to correctly identify fear, it was often confused with surprise.

Whilst on the other hand, the weighted matrices in [Figure 32] show a greater number of misclassifications with weaker diagonalisation. In particular, the Adam optimiser improved performance on the minority classes (fear and sadness) but degraded the classification power of all other emotions; this is further reinforced by the reduced F1-scores in [Figure 33] and [Figure 34]. A phenomenon was observed when reviewing the per-class metrics for the weighted SGD variant, the minority classes performed worse due to fluctuation within the precision and recall scores.

Unweighted Adam Test Loss: 0.6866, Unweighted Adam Test Accuracy: 77.45%

	precision	recall	f1-score	support
Anger	0.7111	0.6400	0.6737	50
Disgust	0.8409	0.7255	0.7789	51
Fear	0.7500	0.3750	0.5000	48
Happiness	0.8750	0.9423	0.9074	52
Sadness	0.6531	0.6667	0.6598	48
Surprise	0.6944	0.9259	0.7937	54
Neurtal	0.8305	0.9333	0.8789	105
accuracy			0.7745	408
macro avg	0.7650	0.7441	0.7418	408
weighted avg	0.7745	0.7745	0.7633	408

Unweighted SGD Test Loss: 0.6454, Unweighted SGD Test Accuracy: 79.66%

	precision	recall	f1-score	support
Anger	0.7500	0.7200	0.7347	50
Disgust	0.8409	0.7255	0.7789	51
Fear	0.6829	0.5833	0.6292	48
Happiness	0.9231	0.9231	0.9231	52
Sadness	0.6327	0.6458	0.6392	48
Surprise	0.7692	0.9259	0.8403	54
Neurtal	0.8716	0.9048	0.8879	105
accuracy			0.7966	408
macro avg	0.7815	0.7755	0.7762	408
weighted avg	0.7956	0.7966	0.7940	408

Figure 33 - Shows the per class metrics for the unweighted loss function, trained with both Adam and SGD

Weighted Adam Test Loss: 0.8431, Weighted Adam Test Accuracy: 67.89%

	precision	recall	f1-score	support
Anger	0.5962	0.6200	0.6078	50
Disgust	0.8214	0.4510	0.5823	51
Fear	0.5000	0.5625	0.5294	48
Happiness	0.7869	0.9231	0.8496	52
Sadness	0.4432	0.8125	0.5735	48
Surprise	0.8333	0.6481	0.7292	54
Neurtal	0.8916	0.7048	0.7872	105
accuracy			0.6789	408
macro avg	0.6961	0.6746	0.6656	408
weighted avg	0.7267	0.6789	0.6844	408

Weighted SGD Test Loss: 0.8676, Weighted SGD Test Accuracy: 72.06%

	precision	recall	f1-score	support
Anger	0.5185	0.8400	0.6412	50
Disgust	0.6964	0.7647	0.7290	51
Fear	0.5714	0.3333	0.4211	48
Happiness	0.8475	0.9615	0.9009	52
Sadness	0.6000	0.4375	0.5060	48
Surprise	0.7541	0.8519	0.8000	54
Neurtal	0.9091	0.7619	0.8290	105
accuracy			0.7206	408
macro avg	0.6996	0.7073	0.6896	408
weighted avg	0.7302	0.7206	0.7128	408

Figure 34 - Shows the per class metrics for the weighted loss function, trained with both Adam and SGD

Unweighted Cross Entropy Loss utilising the updated SGD optimiser was seen as the most effective optimiser and loss function pairing for the updated architecture. It held a greater ability to generalise, outperforming the Adam optimiser across the loss, accuracy and F1-Score metrics. An overall improvement was seen from Architecture 2, particularly with the usage of the unweighted loss function; the weighted variant caused all classes to perform poorly, even if they were in the minority (fear and sadness).

3.5.2.1. Architecture 4:

Even though architecture 3 outperformed architecture 2, mitigating most of its drawbacks, some remained. The architecture was still achieving a loss value which was deemed high; this indicated that misclassification were still occurring, and at a moderate rate. Confusion was apparent between emotions which presented similarly, particularly between fear and surprise.

As a result of this, it was decided that an additional convolutional layer should be added; this final addition hoped to capture features related to the confused emotions, such that they could be easily distinguished. Due to increased layer

depth, overfitting would become apparent once again; a choice was made to implement an additional dropout layer whilst also increasing its probability from 0.4 -> 0.5. Learning rates for each optimiser remained the same, due to their increased performance in Architecture 3. The final architecture is presented in [Figure 35].

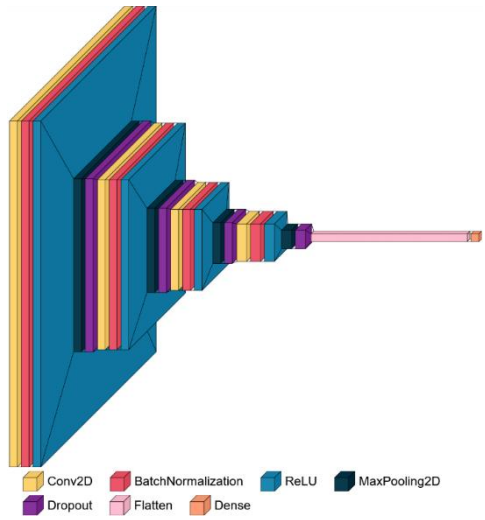


Figure 35 - Shows Architecture 4

Table 14 - Shows the results of training Architecture 4

Optimiser	Loss Function	Accuracy (%)	Loss	F1-Score
Adam	Cross Entropy Loss	67.65	0.8857	0.6035
SGD	Cross Entropy Loss	82.60	0.5441	0.8027
Adam	Weighted Cross Entropy Loss	64.95	1.0160	0.6026
SGD	Weighted Cross Entropy Loss	80.39	0.5838	0.7807

[Table 14] showed the strengths of the SGD optimiser in both weighted and unweighted configurations; it consistently outperformed Adam by approximately 15%. SGD achieved the highest classification accuracy, whilst boasting comparable F1-scores, indicating strong generalisation. Comparing the metrics in [Table 13] and [Table 14] further reinforced the strengths of Architecture 4; the additional convolutional layers aided in the identification of deeper features, dropout reduced the chances of the architecture overfitting to the training data.

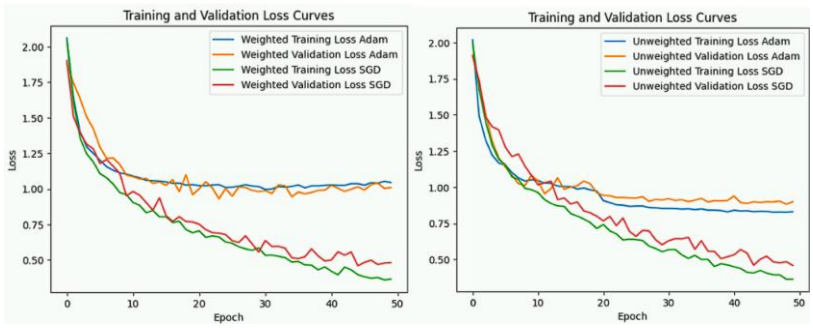


Figure 36 - Shows the loss graphs for Architecture 4

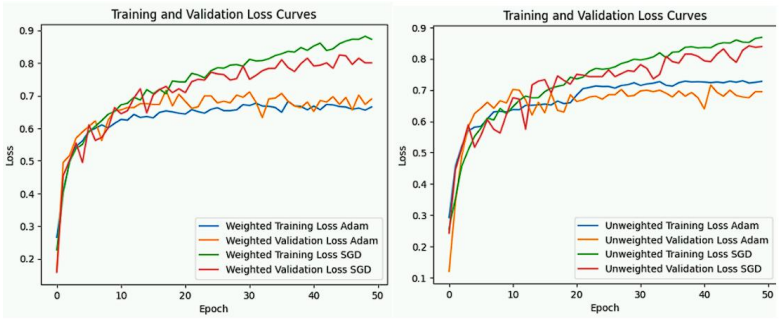


Figure 37 - Shows the accuracy graphs for Architecture 4

Even though the Adam optimiser was performant in the earlier iterations of the architecture, its overall performance decreased after being utilised in both Architecture 3 and 4. An analysis of the loss graph in [Figure 36] shows the difference in each optimiser's performance. In both weighted and unweighted scenarios, SGD's loss curves are continuously decreasing as training occurs, never showing signs of plateauing. Whilst on the other hand, the Adam optimiser's loss curves begin to plateau approximately after the 12th epoch. Both Adam configurations exhibit overfitting, but in different ways: the weighted graphs curves begin to increase after 31st epoch; and the unweighted graphs curves begin to diverge and plateau after the 20th epoch. The accuracy curves in [Figure 37] further reinforce these ideas. SGD's accuracy curves exhibits no signs of stagnation; the curves are still increasing, even though they are not as rapid as seen in the earlier epochs. Adam's accuracy curves show clear stagnation; they struggle to achieve above 70%.

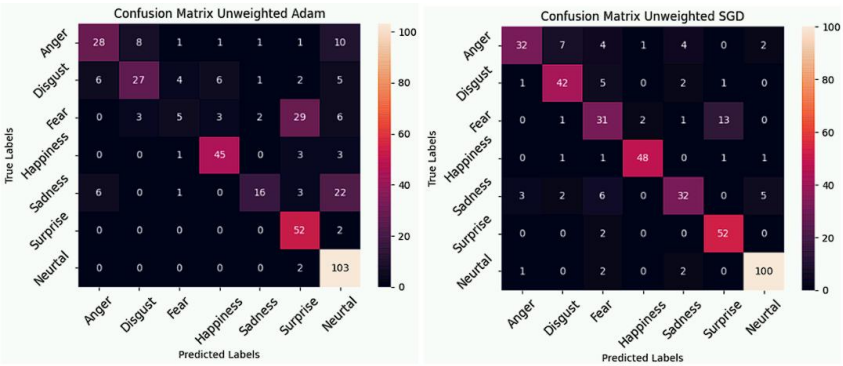


Figure 38 -Shows the confusion matrices for unweighted Adam and SGD

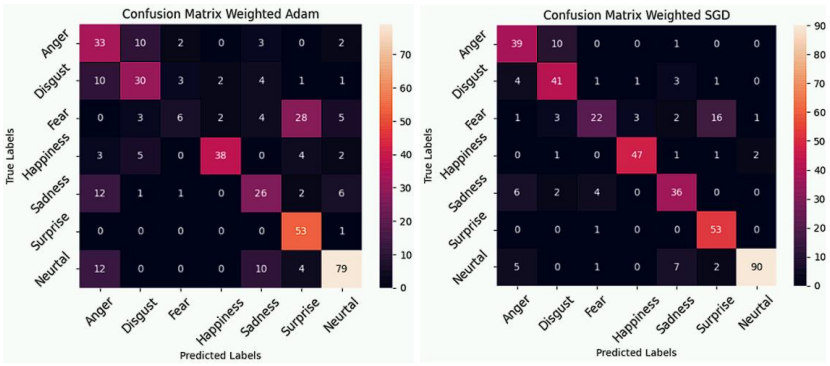


Figure 39 - Shows the confusion matrices for weighted Adam and SGD

Comparing the confusion matrices in [Figure 38] and [Figure 39] further highlights the downfall of the Adam optimiser; both loss configurations with the optimiser led to the loss of diagonalisation. The fear class was only correctly classified 10% (unweighted) and 12.5% (weighted); reviewing the metrics from the previous section highlights a decrease of approximately 27.5% and 44% respectively. Fear was getting confused with surprise, the architecture struggled to differentiate between them. The lack of symmetry on the confusion matrices indicates the architecture learnt how to identify surprise.

SGD performed better than its counter parts from [Figure 31] and [Figure 32] across both weighted and unweighted configurations. Comparing the unweighted configuration in [Figure 38] and [Figure 31] shows the improvement in classification; most classes (excluding anger) saw an increase in correct predictions. Even though fear was still being confused with surprise, it saw a reduction in misclassification with other classes. Reviewing the weighted configurations in [Figure 39] and [Figure 32] shows fear and sadness being classified correctly more often; however, the overall classification accuracy for the remaining classes decreased.

Unweighted Adam Test Loss: 0.8857, Unweighted Adam Test Accuracy: 67.65%

	precision	recall	f1-score	support
Anger	0.7000	0.5600	0.6222	50
Disgust	0.7105	0.5294	0.6067	51
Fear	0.4167	0.1042	0.1667	48
Happiness	0.8182	0.8654	0.8411	52
Sadness	0.8000	0.3333	0.4706	48
Surprise	0.5652	0.9630	0.7123	54
Neurtal	0.6821	0.9810	0.8047	105
accuracy			0.6765	408
macro avg	0.6704	0.6195	0.6035	408
weighted avg	0.6724	0.6765	0.6356	408

Unweighted SGD Test Loss: 0.5441, Unweighted SGD Test Accuracy: 82.60%

	precision	recall	f1-score	support
Anger	0.8649	0.6400	0.7356	50
Disgust	0.7925	0.8235	0.8077	51
Fear	0.6078	0.6458	0.6263	48
Happiness	0.9412	0.9231	0.9320	52
Sadness	0.7805	0.6667	0.7191	48
Surprise	0.7761	0.9630	0.8595	54
Neurtal	0.9259	0.9524	0.9390	105
accuracy			0.8260	408
macro avg	0.8127	0.8021	0.8027	408
weighted avg	0.8293	0.8260	0.8236	408

Figure 40 - Shows the per class metrics for the unweighted loss function, trained with both Adam and SGD

Weighted Adam Test Loss: 1.0160, Weighted Adam Test Accuracy: 64.95%

	precision	recall	f1-score	support
Anger	0.4714	0.6600	0.5500	50
Disgust	0.6122	0.5882	0.6000	51
Fear	0.5000	0.1250	0.2000	48
Happiness	0.9048	0.7308	0.8085	52
Sadness	0.5532	0.5417	0.5474	48
Surprise	0.5761	0.9815	0.7260	54
Neurtal	0.8229	0.7524	0.7861	105
accuracy			0.6495	408
macro avg	0.6344	0.6256	0.6026	408
weighted avg	0.6615	0.6495	0.6318	408

Weighted SGD Test Loss: 0.5838, Weighted SGD Test Accuracy: 80.39%

	precision	recall	f1-score	support
Anger	0.7091	0.7800	0.7429	50
Disgust	0.7193	0.8039	0.7593	51
Fear	0.7586	0.4583	0.5714	48
Happiness	0.9216	0.9038	0.9126	52
Sadness	0.7200	0.7500	0.7347	48
Surprise	0.7260	0.9815	0.8346	54
Neurtal	0.9677	0.8571	0.9091	105
accuracy			0.8039	408
macro avg	0.7889	0.7907	0.7807	408
weighted avg	0.8134	0.8039	0.8003	408

Figure 41 -Shows the per class metrics for the unweighted loss function, trained with both Adam and SGD

Reviewing the per class metrics in [Figure 40] and [Figure 41] further highlights the strengths of the unweighted SGD configuration; [Figure 40] shows an overall increase in F1-Score across all classes. Even though precision saw some decrease from [Figure 33] (especially within the disgust and fear classes), the recall ability was improved; the architecture gained an understanding of the classes, knowing when an image was not related to a certain class. This was also seen when a comparison was made between its weighted configuration in [Figure 34]; all classes had their F1-Scores increased, improving in both precision and recall capabilities.

Adam's per class metrics saw an overall decrease in performance, especially after being paired with the weighted loss function. Comparing the current unweighted configuration in [Figure 40] to the previous in [Figure 33] revealed a decrease in all F1-Scores; the fear class was the most affected, decreasing by almost 67% from 0.5000 to 0.1667. The weighted configuration displayed similar results; although performance on the minority classes saw some improvement, such that their F1-scores increased from [Figure 34] to [Figure 41].

Once again, the unweighted cross entropy loss paired with the SGD optimiser was seen as the most effective pairing for the fourth iteration of the architecture. The weighted loss function achieved similar performance, the sadness class reacted positively towards the weighting; however, the fear class did not. Overall, the weighted variant caused greater instability after the addition of another convolutional layer and stronger regularisation.

Conclusion:

This section investigated the effects of using a newly developed architecture to perform FER, several techniques were utilised to further improve the optimisation of the architecture's weights. This was achieved using a combination of different optimisers (Adam and SGD) and loss functions (unweighted and weighted cross entropy loss). Architectural alterations were also made; for example, the addition of more layers. Convolutional and regularisation layers were added after each iteration, in hopes of extracting deeper features and preventing overfitting.

The final iteration resulted in four architectures, the best performing was trained using unweighted cross entropy loss with the SGD optimiser; it achieved an accuracy of 82.60% and an F1-score of 0.8027. The training phase exhibited smooth converge, with a negligible amount of overfitting; training could be further extended as the curves never plateaued. It performed exceptionally on the neutral and happy classes, achieving F1-scores above 0.9000. Performance on the minority classes was varied; the sadness class saw an increase in its F1-score from 0.6392 -> 0.7347 but fear saw a decrease from 0.6292 -> 0.6263. The architecture's performance saw improvements in generalisability and convergence, as layers were added. Overfitting was mitigated by the fourth iteration; the regularisation techniques prevent the architecture from memorising the training data. Comparing [Figure 20] to [Figure 36] showed the reduction in curve divergence, the architecture became complex enough to learn the features whilst also remaining moderate in depth.

Earlier iterations of the architecture favoured the Adam optimiser, it converged the fastest and achieved the highest accuracies in comparison to the other configurations; in addition, it also produced a model which had a strong capability to generalise (high F1-Score). However, as the architecture developed further, the overall performance of the optimiser dropped. Even though it remained stable during training, the overall loss was higher compared to SGD; the architecture was getting penalised more often for misclassification. As a result, its F1-score and accuracy saw a drop. SGD began to outperform Adam after having its hyper-parameters altered in Architecture 3; the loss curves in later iterations exhibited none-little overfitting and converged at a lower loss value than Adam.

To address the class imbalance, two loss functions were introduced: weighted cross entropy loss, and cross entropy loss. Weighted cross entropy loss attempted to improve performance in the minority classes by applying harsher weighting factors of loss to them. The performance of the weighted loss function varied throughout testing; earlier revisions of the architecture showed its strengths. The minority classes saw improved classification performance; the architecture was penalising itself enough such that each class was getting treated equally. However, the converse was shown as the architecture developed; the addition of more layers and stronger regularisation resulted in over-penalisation. Later revisions of the architecture were able to extract and understand deeper features, such that the weights were already optimised; the weighted loss function applied excessive scaling to these weights, resulting in a greater level of confusion during classification.

To conclude this section, the creation of the custom architecture with optimisation techniques was an overall success; an architecture was created which performed FER strongly, only slightly struggling to classify some of the minority classes (i.e., sadness). The choice of optimiser largely affected the architecture's ability to learn and converge, SGD enabled stabler training, producing some of the highest results. The type of loss function used did not improve performance greatly, it often limited the architecture's capability to learn.

4 Evaluation:

The project tested three strategies for performing FER. The first and second approaches utilised transfer learning, unfreezing the fully connected/classifier layer and unfreezing additional earlier convolutional layers. The architectures were trained on the individual datasets and then trained using the compiled dataset. The third approach involved the creation of a custom CNN architecture which was optimised using different loss functions and optimisers. The goal of these strategies was to compare the classification and generalisability performance of a freshly trained architecture against a pre-trained counterpart.

4.1. Results of the tested approaches:

[Table 15] displays the best performing architecture in each approach listed above; some approaches have more than one listing, due to their strong accuracy/F1-score.

Table 15 - Shows the best performing models across all approaches
CEL – Cross Entropy Loss / WCEL – Weighted Cross Entropy Loss

ID	Architecture	Approach	Training Dataset	Testing Dataset	Accuracy (%)	Loss	F1-Score	Overfitting (None/Minimal/Moderate/High)
1	ResNet-18	1	KDEF	KDEF	53.74	1.4090	0.5202	None
2	ResNet-18	2	CK+	CK+	91.40	0.2914	0.7503	Moderate
3	ResNet-18	2	KDEF	KDEF	90.14	0.2981	0.9004	Moderate
4	ResNet-18	2	Compiled	Compiled	89.46	0.3288	0.8813	Moderate
5	Architecture 4 (CEL + SGD)	3	Compiled	Compiled	82.60	0.5441	0.8027	None
6	Architecture 4 (WCEL + SGD)	3	Compiled	Compiled	80.39	0.5838	0.7807	Minimal
7	DeepFace	-	Compiled	Compiled	47.05	-	-	-

4.2. Evaluation of approaches

The results in [Table 15] compare the effects of using transfer learning against a freshly trained custom architecture. The initial approach to transfer learning achieved the worst scores in terms of accuracy and F1, even though there were no apparent signs of overfitting. This demonstrated the limitations of attempting to perform FER when only the fc layer is unfrozen; the architecture is unable to learn generalisable features to their full extent due to a constraint on lack of trainable parameters. This was seen throughout [Section 3.4.5.1].

After earlier convolutional layers were unfrozen in addition on to those stated in approach 1, the accuracy and F1-scores saw major improvement. Due to an increase in unfrozen layers, the number of trainable parameters saw an increase; these additional parameters could be used to retain deeper features which were not learnt by the previous approach (ID 1-2). Even though strong performance was exhibited (an increase in accuracy and generalisability), the architectures struggled immensely after being exposed to new testing data, as seen in [Section 3.4.5.2 – Cross-set Generalisability]. [Table 9] displayed poor performance across every architecture; the accuracy and F1-Score ranged between (7%-40%)

and (0.0443 – 0.4007) respectively. Even though the F1 metrics depict some level of generalisability, it was dataset specific, rather than globally specific.

To improve generalisability, the datasets were compiled and used to train the three architectures. The best performing architecture in this scenario was ResNet-18 – achieving 89.46% accuracy and an F1-score of 0.8813 (ID 4). Even though the accuracy and F1-score saw a decrease within their values, the architecture's overall ability to learn features theoretically improved. As a greater level of diversity and variance was introduced into the training set, there was no long a strong skew to a singular ethnicity (i.e., JAFFE – only Japanese Females). Despite its introduction, the architecture exhibited moderate signs of overfitting during the training process; this was most likely due to the combination of the ResNet-18's depth, and the relatively small size of the dataset.

Architecture 4 (ID 5-6) exhibited less stability during the training process but showed no-minimal signs of overfitting [Figure 36]; this was apparent in both weighted and unweighted configuration when using the SGD optimiser. Even though the custom architecture did not surpass the performance of the transfer learning architectures both in terms of accuracy and F1-score, it was viewed as being more robust. [Table 15] shows the transfer learning architectures overfitting moderately; dataset specific features were being memorised, whilst generalisable features were being overlooked.

Throughout [Section 3.5], both weighted and unweighted cross entropy loss were tested; the experiment wanted to test their impact on an imbalanced dataset. [Section 3.5 – Conclusion] discussed the benefits of the weighted variant especially within shallower architectures, and the drawbacks of its usage within deeper architectures. Earlier scenarios lacked regularisation and meaningful feature extraction hence why the weighted loss function outperformed its counterpart. Whilst within the final scenario, the unweighted loss function was seen as advantageous; the final iteration of the architecture possessed enough power to extract and learn features from all classes whether they were over/under-represented.

[Section 3.5] highlighted the advantages of using Adam during the earlier developments of architecture; it converged at a faster rate than SGD. However, as the architecture saw further development, Adam became less effective. Convergence happened before SGD, but occurred at a local minimum, rather than a global one; SGD optimised further than Adam, indicating the model had greater capacity to learn. For example, in Architecture 4, SGD achieved an accuracy 15% better than Adam (Adam: 67.65%, SGD: 82.60%), whilst also achieving an F1-score approximately 33% higher (Adam: 0.6035, SGD: 0.8027). SGD with momentum allowed for the global minimum to be located, enabling for optimisation to be continued further than Adam.

Reviewing the respective confusion matrices for [ID 2-6] highlights a common trend which all architectures followed; all showed some level of confusion between surprise and fear, such that fear was predicted as surprise. The same can be said for the confusion matrix in [5], it too struggled with differentiating between fear and surprise. However, this was not apparent in DeepFace's confusion matrix [Figure 2]; the architecture that DeepFace used to perform FER holds a greater ability to distinguish the difference between fear and surprise than the other architectures. This is most likely due to the type of training data which was used by DeepFace during its training.

[Table 15] depicts the performance of DeepFace; the finely tuned transfer learning and custom architectures (ID 4-5) both outperformed DeepFace substantially. They achieved an accuracy which was approximately x2 that of DeepFace. However, the comparison is not truly fair; [Section 2.4.2] states DeepFace's primary purpose is for facial detection. The DeepFace model has not been optimised for FER; rather for locating faces within an image and then detecting whether they match them to occurrences within a defined folder/database.

4.3. Further work

Even though the project was an overall success, such that the effects of optimisation were evaluated using both a transfer learning and custom architectural approach, there was still room further development due to several limitations within the project.

The most prominent limitation within this project was the reliance on relatively small, controlled datasets; for example, the JAFFE and KDEF datasets. As previously mentioned, these datasets lacked diversity, only representing a singular demographic (e.g., JAFFE - Japanese Females). [Section 3.4.5.2 – Cross-set Generalisability] highlighted the biases which were introduced when these datasets were used for training. The finely tuned architectures were unable to generalise to new data, struggling to identify features.

Another limitation was the lack of regularisation when testing the transfer learning approaches mentioned in [Section 3.4.5.2], especially in the compiled section. The architectures commonly overfitted towards the training set, producing inflated results. The custom architecture implemented regularisation techniques such as dropout and weight decay in attempt to reduce the effects of overfitting. If testing were to be re-evaluated, these regularisation techniques would also be applied to the transfer learning approaches; this would help create a more balance argument during the comparative analysis of all approaches.

To further develop the study and reduce the chances of creating a biased architecture, a larger dataset with greater diversity (i.e., RAF-DB, FER-2013, AFFECT-NET) could be used. Previous studies such as [5] and [13] highlighted the advantages of using these datasets; their architectures achieved high accuracies and strong performing metrics. More training data would help create an architecture which could learn generalisable features to a greater extent. In addition to diversity, a greater variance would also be introduced; images within the dataset would be taken in different environments. This would help improve the architecture's robustness within a real-world setting.

Both extension objectives mentioned in [Section 1] were not achieved: using Grad-CAM to visualise feature map analysis; and implementing real -time FER using a webcam or video. The real-time implementation was hindered by ethical constraints; additional approval would have been required to have subjects test the application. Even though Grad-CAM would have provided an insight into how the architectures were reacting to the images during the testing process, its implementation was not achieved due to time constraints. Further work would involve achieving these objectives.

5 LSEP Section:

5.1. Ethical Approval:

The project was sent through the universities' Ethics RM approval system, where it received approval.

5.2. Data Usage:

This project used datasets specific to FER, such that they contained images of participants expressing a range of emotions. The images fall under the secondary data classification; this project did not involve the collection of additional personal data (i.e., name, contact information, address, additional images). The datasets are publicly available, and sourced from Kaggle [17], [18, [19]. Each dataset had a set of valid usage rights and licensing rules; for example, KDEF and CK+ both were distributed under the CC0: Public Domain license [30]. The license states that the dataset can be used in manor, whether it is copied, modified or distributed. Datasets such as JAFFE and KDEF contained labelled images, separated into folders; CK+ contained pixel values which had to be extracted and turned into images. The participants gave consent to be included within the dataset [31], [32] and (implied by) [33].

The data within these datasets was only used to train the architectures and test the existing solution (DeepFace); the data had no other purpose. It was stored locally on the universities secure system; authentication was required to access the data due to it residing in an individual workspace environment. [Section 3.3] highlighted the use of Google Colab (an online programming environment) to test DeepFace; the dataset was uploaded to its virtual environment from the individual workspace. The dataset remained within the workspace until the code completed execution; afterwards, it was deleted from the environment.

[Section 3.1.6] highlighted several datasets which were popular amongst other research papers; however, these datasets were excluded from this project. The datasets included images which were scrapped from the internet; images within the datasets were taken without the participant's formal consent. In addition, some of these datasets included participants who were under the age of 18; their images could have been used without their guardian's formal consent.

5.3. Ethical Challenges:

Due to the nature of the project and the data used, there were some challenges which were faced that required consideration if the project were to be deployed in a real-time environment. For examples, the introduction of ethnic bias and lack of representation; the JAFFE dataset contained only images of Japanese Females, whilst both KDEF and CK+ primarily contained Caucasians. This risked the creation and development of an architecture which would struggle to identify emotions to those who are of a different ethnicity. The dataset compilation strategy from [Section 3.4.5.2] was used to mitigate the risk; however, it was still prevalent even though diversity grew.

An additional challenge was the over-representation of certain emotions (i.e., neutral), this introduced a class imbalance. The CK+ dataset showed fear having 25 samples, whilst neutral had 593. This introduced a bias, the architectures overfitted towards the majority class, reducing its ability to identify other emotions. Mitigation strategies were discussed in [Section 3.1]; however, the majority (i.e., SMOTE and Oversampling) were discarded due to increase in the previously mentioned bias.

5.4. Relevance to SDG:

While the current version of the project does not align with any SDG goals, it could have the potential to contribute to them if further developments are made. If the current solution had the extension feature of performing FER within a real-time environment , it could be used within a psychological setting to determine a person's mental health; this would align with SDG 3 – Good Health and Well-Being [34].

References:

[1]
IBM, “Computer Vision,” *IBM*, 2019. https://www.ibm.com/topics/computer-vision

[2]
“VERBAL | meaning in the Cambridge English Dictionary,” *dictionary.cambridge.org*. https://dictionary.cambridge.org/dictionary/english/verbal

[3]
Cambridge Dictionary, “COMMUNICATION | meaning in the Cambridge English Dictionary,” *Cambridge.org*, 2024. https://dictionary.cambridge.org/dictionary/english/communication

[4]
“NON-VERBAL | meaning in the Cambridge English Dictionary,” *dictionary.cambridge.org*. https://dictionary.cambridge.org/dictionary/english/non-verbal

[5]
werywjw, “SEP-CVDL/final/GIMEFIVE: Towards Interpretable Facial Emotion Classification.pdf at main · werywjw/SEP-CVDL,” *GitHub*, 2023. https://github.com/werywjw/SEP-CVDL/blob/main/final/GIMEFIVE%3A%20Towards%20Interpretable%20Facial%20Emotion%20Classification.pdf

[6]
P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” *ACCEPTED CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION*, 2001, Available: https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf

[7]
Honey and Sukhwinder Singh Oberoi, “Revolutionizing Face Detection: Exploring the Potential of MTCNN Algorithm for Human Face Recognition,” Nov. 2023, doi: https://doi.org/10.1109/iciip61524.2023.10537691.

[8]
Sumit Tariyal, R. Chauhan, Yogesh Bijalwan, R. Rawat, and R. Gupta, “A comparitive study of MTCNN, Viola-Jones, SSD and YOLO face detection algorithms,” Jan. 2024, doi: https://doi.org/10.1109/iitcee59897.2024.10467445.

[9]
F. Z. Canal *et al.*, “A survey on facial emotion recognition techniques: A state-of-the-art literature review,” *Information Sciences*, vol. 582, pp. 593–617, Jan. 2022, doi: https://doi.org/10.1016/j.ins.2021.10.005.

[10]
GeeksforGeeks, “Introduction to Support Vector Machines (SVM),” *GeeksforGeeks*, Jun. 09, 2020. https://www.geeksforgeeks.org/introduction-to-support-vector-machines-svm/?ref=next_article

[11]
GeeksforGeeks, “Image classification using Support Vector Machine (SVM) in Python,” *GeeksforGeeks*, Mar. 21, 2023. https://www.geeksforgeeks.org/image-classification-using-support-vector-machine-svm-in-python/

[12]
H. I. Dino and M. B. Abdulrazzaq, “Facial Expression Classification Based on SVM, KNN and MLP Classifiers,” *2019 International Conference on Advanced Science and Engineering (ICOASE)*, Apr. 2019, doi: https://doi.org/10.1109/icoase.2019.8723728.

[13]
M. C. GURSESLI, S. LOMBARDI, M. DURADONI, L. BOCCHI, A. GUAZZINI, and A. LANATA, “Facial Emotion Recognition (FER) Through Custom Lightweight CNN Model: Performance Evaluation in Public Datasets,” *Ieee.org*, 2024. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10477992

[14]
S. I. Serengil, “serengil/deepface,” *GitHub*, Aug. 30, 2020. https://github.com/serengil/deepface

[15]
Sefik Serengil, “Facial Expression Recognition with Keras - Sefik Ilkin Serengil,” *Sefik Ilkin Serengil*, 2018. https://sefiks.com/2018/01/01/facial-expression-recognition-with-keras/

[16]
M. Z. Alom *et al.*, “The History Began from AlexNet: A Comprehensive Survey on Deep Learning Approaches,” *arXiv:1803.01164 [cs]*, Sep. 2018, Available: https://arxiv.org/abs/1803.01164

[17]
D. Sena, “CK+ Dataset,” *Kaggle.com*, 2023. https://www.kaggle.com/datasets/davilsena/ckdataset

[18]
tarunparuchur, “JAFFE Image Dataset,” *Kaggle.com*. https://www.kaggle.com/datasets/tarunparuchur/jaffe-image-dataset

[19]
C. Rich, “KDEF Database,” *Kaggle.com*, 2025. https://www.kaggle.com/datasets/chenrich/kdef-database

[20]
Tamanna, “Handling Imbalanced Datasets in Python: Methods and Procedures,” *Medium*, Oct. 18, 2023. https://medium.com/@tam.tamanna18/handling-imbalanced-datasets-in-python-methods-and-procedures-7376f99794de

[21]
N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, no. 16, pp. 321–357, Jun. 2002, doi: https://doi.org/10.1613/jair.953

[22]
J. Brownlee, “Random Oversampling and Undersampling for Imbalanced Classification,” *Machine Learning Mastery*, Jan. 14, 2020. https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/

[23]
hengtao tantai, “Use weighted loss function to solve imbalanced data classification problems,” *Medium*, Feb. 27, 2023. https://medium.com/@zergtant/use-weighted-loss-function-to-solve-imbalanced-data-classification-problems-749237f38b75

[24]
Sardor Abdirayimov, “Face Recognition in Python | Part 1 | FaceNet, MTCNN, SVM,” *YouTube*, Dec. 27, 2022. https://www.youtube.com/watch?v=bG2tNYs7gw8

[25]
S. Abdirayimov, “main,” *Google.com*, 2024. https://colab.research.google.com/drive/1TjgM1Uy9TIUk091cAmLEnqk9XqUtoqUF#scrollTo=9U_uHILx0Bd8

[26]
Ragon Ebker, “Baeldung,” *Baeldung on Computer Science*, Mar. 15, 2022. https://www.baeldung.com/cs/transfer-learning-vs-domain-adaptation

[27]
M. Yaqub *et al.*, “State-of-the-Art CNN Optimiser for Brain Tumor Segmentation in Magnetic Resonance Images,” *Brain Sciences*, vol. 10, no. 7, p. 427, Jul. 2020, doi: https://doi.org/10.3390/brainsci10070427.

[28]
S. Gupta, “Common Loss Functions in Machine Learning | Built In,” *builtin.com*. https://builtin.com/machine-learning/common-loss-functions

[29]
“Pytorch Dataloader Workers Explained | Restackio,” *Restack.io*, 2024. https://www.restack.io/p/pytorch-answer-dataloader-workers

[30]
Creative Commons, “Creative Commons — CC0 1.0 Universal,” *Creativecommons.org*, 2019. https://creativecommons.org/publicdomain/zero/1.0/

[31]
M. G. Calvo, A. Fernández-Martín, G. Recio, and D. Lundqvist, “Human Observers and Automated Assessment of Dynamic Emotional Facial Expressions: KDEF-dyn Database Validation,” *Frontiers in Psychology*, vol. 9, Oct. 2018, doi: https://doi.org/10.3389/fpsyg.2018.02052.

[32]
M. Lyons, “Indiscretions of a Contemporary Artist - Michael Lyons - Medium,” *Medium*, Sep. 23, 2021. https://medium.com/%40michael.lyons_85617/indiscretions-of-a-contemporary-artist-88c9528a3ec1

[33]
P. Lucey, J. F. Cohn, T. Kanade, J. Saragih, Z. Ambadar, and I. Matthews, “The Extended Cohn-Kanade Dataset (CK+): A complete dataset for action unit and emotion-specified expression,” *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition - Workshops*, Jun. 2010, doi: https://doi.org/10.1109/cvprw.2010.5543262.

[34]
United Nations, “Goal 3: Ensure healthy lives and promote well-being for all at all ages,” *United Nations*, 2015. https://sdgs.un.org/goals/goal3